



Software Design and Analysis Project Report

CodeArena

A Local JavaFX Competitive Coding Arena

Prepared by	Zeeshan Azeem (24I-0588) Sharjeel Ali Khan (24I-0632) Abdul Kabeer (24I-0802)
Course Area	Software Design and Analysis
Technology Stack	Java 17, JavaFX 21, Maven, SQLite, BCrypt, ProcessBuilder
Date	2026-05-07

Document Purpose

This report documents requirements, analysis, design, architecture, database behavior, user interface, testing evidence, and deployment view of the implemented CodeArena desktop application.

Index

The index below is generated from Word heading fields.

Index.....	2
1. Introduction.....	3
1.1 Purpose.....	3
1.2 Product Scope.....	3
1.3 Title.....	3
1.4 Objectives.....	3
1.5 Problem Statement.....	3
2. Overall Description.....	3
2.1 Product Perspective.....	3
2.2 Product Functions.....	3
2.3 List of Use Cases.....	4
2.4 Extended Use Cases.....	4
2.5 Use Case Diagram.....	5
3. Other Nonfunctional Requirements.....	5
3.1 Performance Requirements.....	5
3.2 Safety Requirements.....	5
3.3 Security Requirements.....	5
3.4 Software Quality Attributes.....	5
3.5 Business Rules.....	6
3.6 Operating Environment.....	6
3.7 User Interfaces.....	6
4. Domain Model.....	9
5. System Sequence Diagram.....	9
6. Sequence Diagram.....	10
7. Class Diagram.....	11
8. Package Diagram.....	12
9. Deployment Diagram.....	12
10. Database Design and Persistence.....	13
11. Testing and Validation.....	13
12. Conclusion.....	13

1. Introduction

1.1 Purpose

This report describes the requirements, analysis, design, implementation structure, and validation of CodeArena. It follows the provided Software Requirements and Design Document template and adapts it for a Software Design and Analysis project submission.

1.2 Product Scope

CodeArena is a local competitive programming desktop application. It supports account management, problem solving, local code judging, gamified progress, battles, squads, leaderboards, admin content management, and visual progress tracking.

1.3 Title

CodeArena: A Local JavaFX Competitive Coding Arena

1.4 Objectives

- Provide a complete local coding practice platform with login, registration, problem browsing, and submissions.
- Judge Java and Python solutions locally using ProcessBuilder and isolated temporary work directories.
- Use XP, ranks, streaks, badges, leaderboards, and battle records to encourage continued practice.
- Support private 1v1, random 1v1, and free-for-all battle workflows.
- Provide an admin panel for managing users, problems, test cases, submissions, and analytics.
- Maintain a clear package-based architecture with UI, service, DAO, model, judge, and utility layers.

1.5 Problem Statement

Students often need a compact environment where they can practice coding problems, receive immediate feedback, and compete with classmates without depending on external competitive-programming platforms. Manual checking of solutions is slow, inconsistent, and does not scale well when multiple users submit code.

CodeArena solves this problem by combining local judging, progress tracking, badges, leaderboards, battles, squads, and admin controls in one JavaFX desktop application.

2. Overall Description

2.1 Product Perspective

CodeArena is a standalone local desktop application. It starts from MainApp, initializes persistence, displays a splash screen, and then opens the login interface. It stores data in a local SQLite database and builds screens with JavaFX.

2.2 Product Functions

- Register and authenticate users with BCrypt password hashing.
- Allow guests to browse problems and view the public leaderboard.
- Display problem lists, filters, details, sample test cases, and acceptance rates.
- Run and submit Java/Python code through the local judge.
- Store submissions, verdicts, runtime, XP updates, badges, and battle records.
- Support dashboards, profile charts, submission history, battle history, and achievements.
- Provide admin features for users, problems, test cases, submissions, and analytics.
- Initialize SQLite schema and seed data automatically when needed.

2.3 List of Use Cases

- Register Account
- Login
- Browse Problems
- View Problem Details
- Run Code
- Submit Solution
- View Profile
- Earn Badge
- Start Battle
- Join Battle
- Create/Join Squad
- View Leaderboard
- Admin Manage Users
- Admin Manage Problems
- Admin Review Submissions

2.4 Extended Use Cases

Use Case	Primary Actor	Description
Submit Solution	Coder	Coder writes Java/Python code, submits it, and receives a stored verdict after the local judge evaluates test cases.
Create Battle Room	Coder	Coder creates a battle room, shares the code, and competes once another user joins.
Admin Create Problem	Admin	Admin enters problem metadata, statement, constraints, input/output format, and sample/hidden tests.
View Profile Progress	Coder	Coder reviews verdict charts, language usage, battle stats, history, and achievements.

2.5 Use Case Diagram

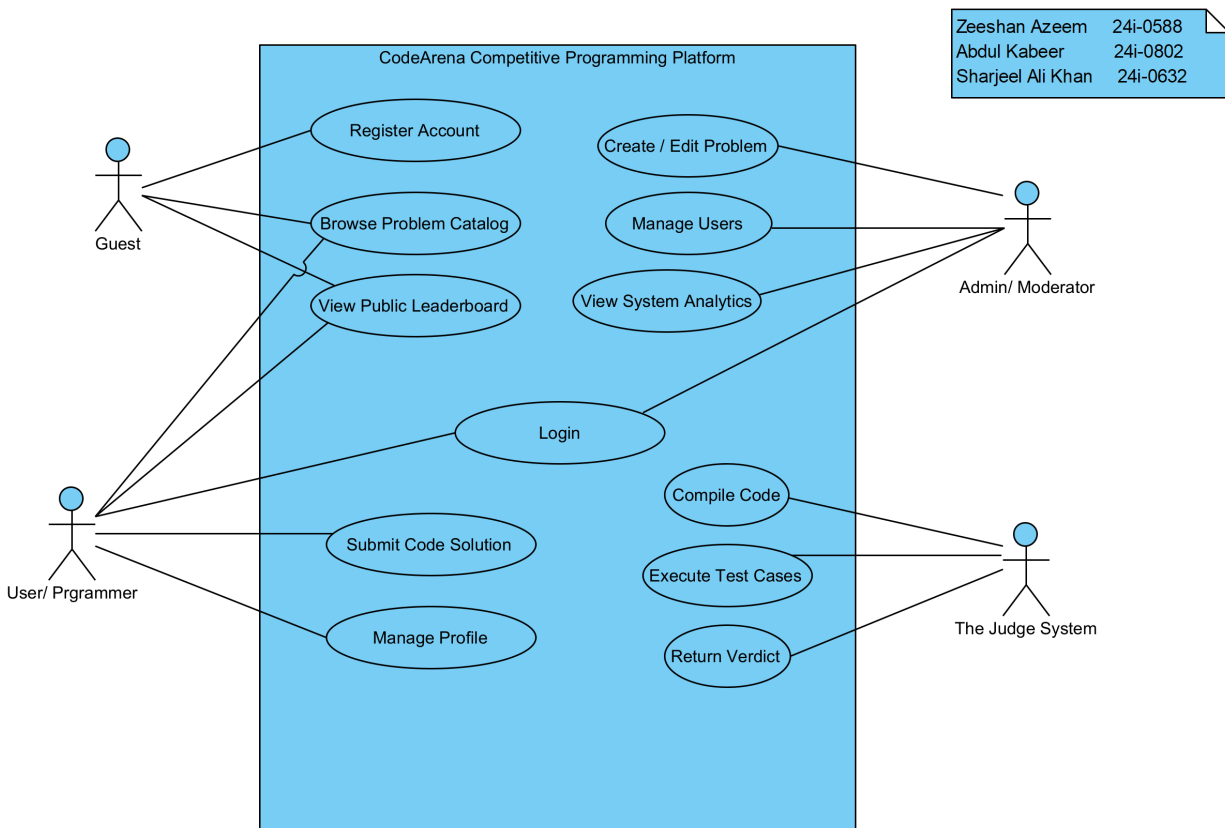


Figure 1: Use Case Diagram

3. Other Nonfunctional Requirements

3.1 Performance Requirements

- Screens should load quickly using local SQLite data.
- Judge execution must run outside the JavaFX UI thread.
- Infinite loops must be terminated through time-limit handling.
- Temporary judge files must be cleaned after each run.

3.2 Safety Requirements

- Submissions run in temporary OS directories.
- Destructive admin operations require confirmation.
- Runtime files are cleaned after AC, WA, CE, RE, and TLE outcomes.

3.3 Security Requirements

- Passwords are stored using BCrypt hashes.
- Admin pages require an authenticated ADMIN role.
- Inactive/banned users cannot log in.
- SQL is isolated in DAO and persistence classes.

3.4 Software Quality Attributes

- Maintainable package structure.
- Reliable schema/seed initialization.
- Usable dark/light interface with dashboards and tables.

- Testable service, persistence, and judge behavior.
- Extensible problem, badge, battle, and squad systems.

3.5 Business Rules

- Only active users can log in.
- Admin accounts cannot be banned from the admin panel.
- Problems require valid metadata and test cases before saving.
- Accepted submissions may award XP and badges.
- Battle results update user records.
- Squad ranking uses combined XP.

3.6 Operating Environment

- Java 17 or later.
- Maven 3.8 or later.
- JavaFX 21 dependencies through Maven.
- SQLite JDBC.
- Optional Python runtime for Python submissions.
- Windows desktop environment for demonstrated screenshots.

3.7 User Interfaces

The UI uses JavaFX screens, a shared sidebar, themed panels, charts, tables, dialogs, and form controls. Styling is centralized in a JavaFX CSS file while screen composition and event handling remain in Java code.



Figure: Splash Screen

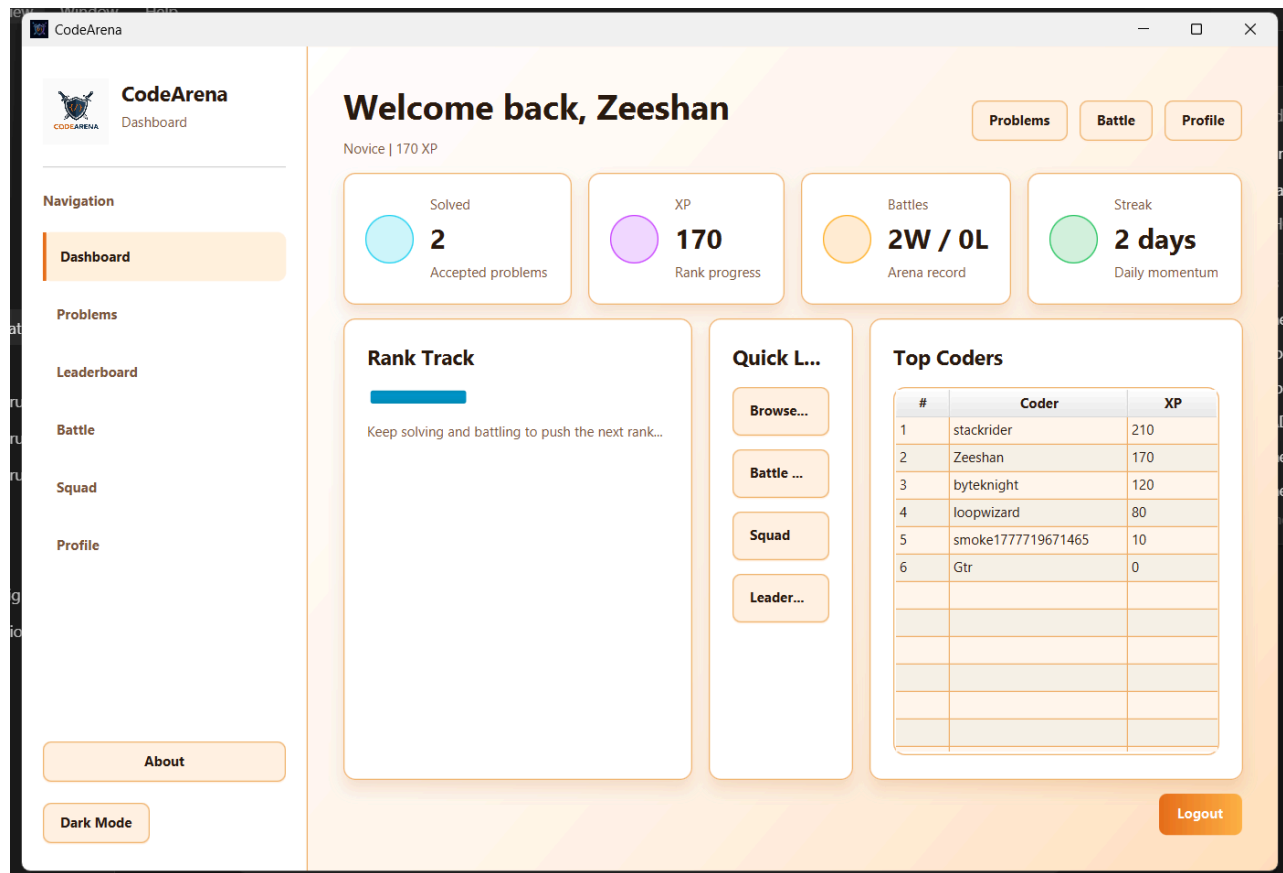


Figure: Coder Dashboard

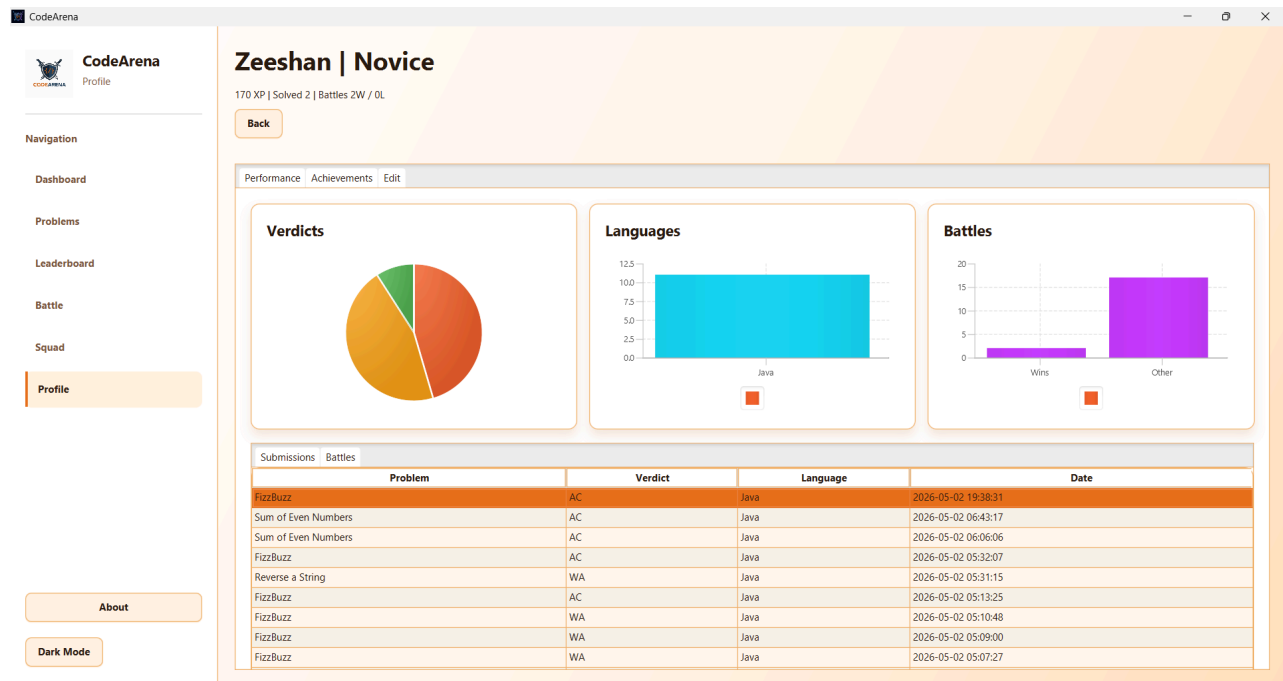


Figure: Profile Performance View

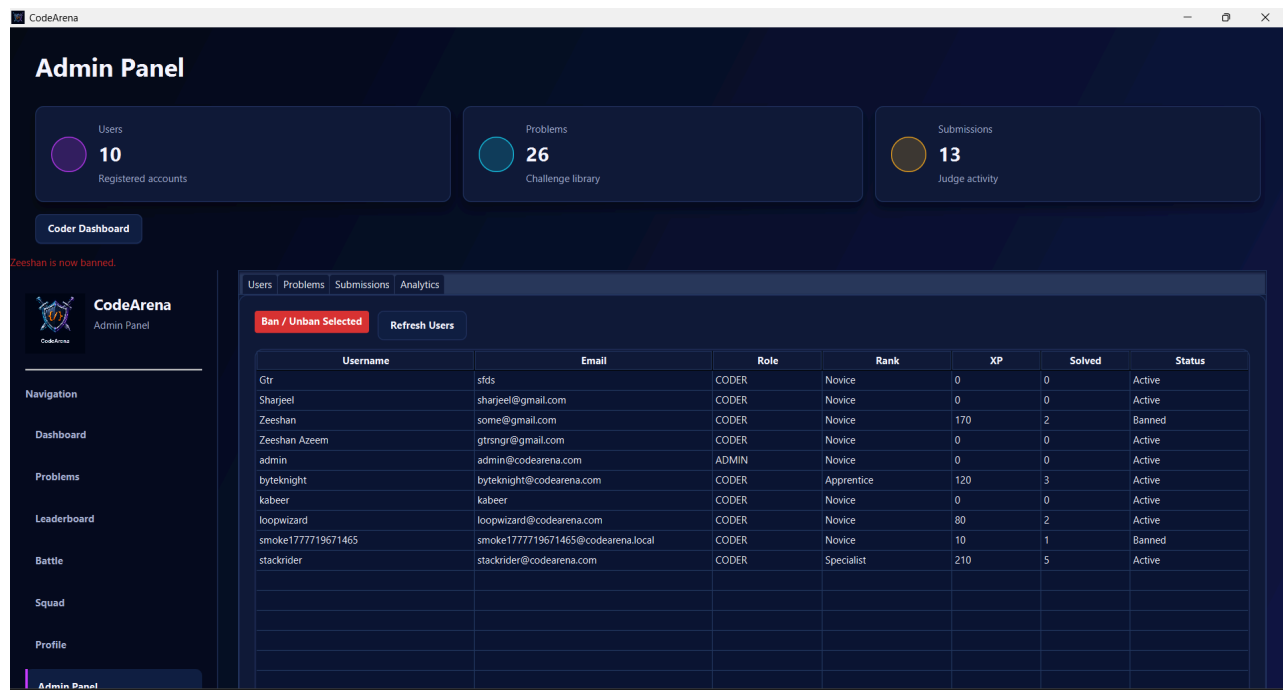


Figure: Admin Panel

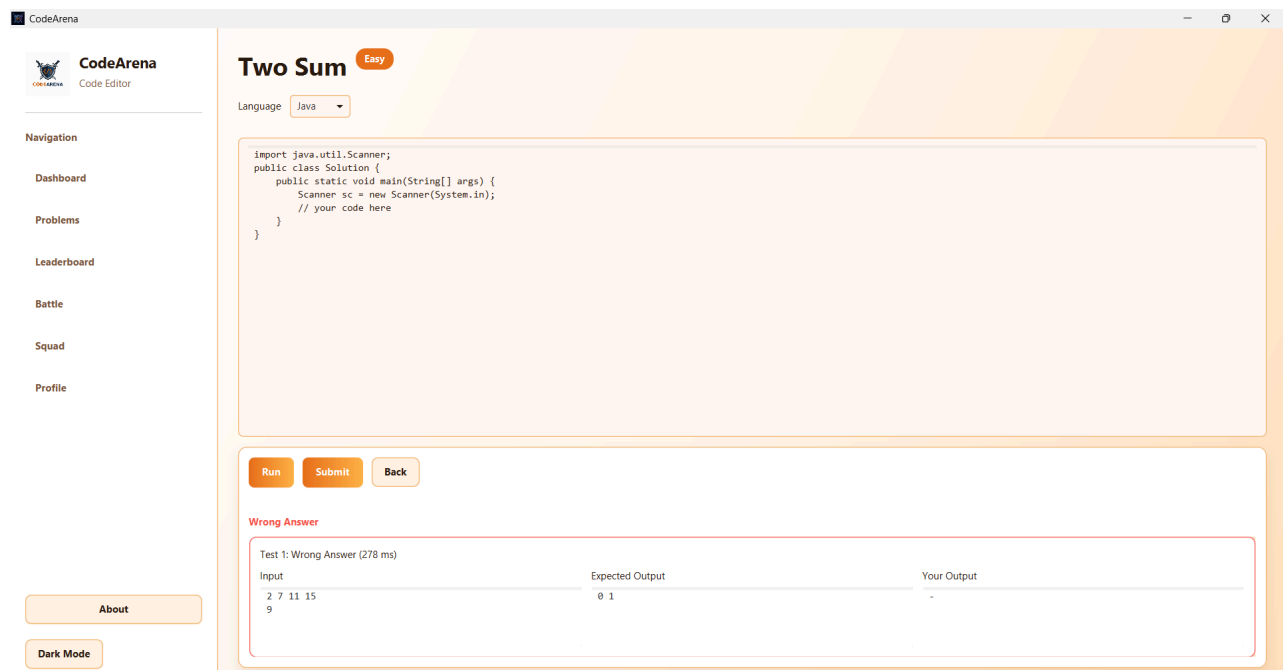


Figure: Code Editor and Judge Output

4. Domain Model

Domain Model

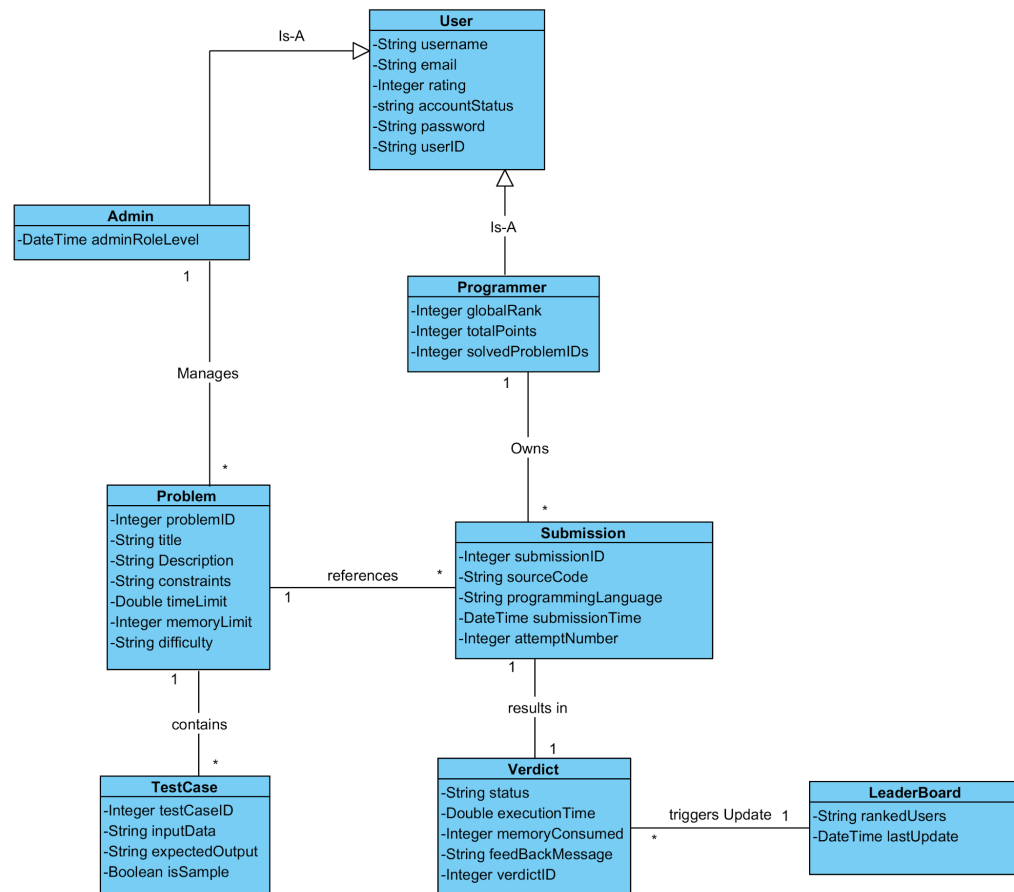
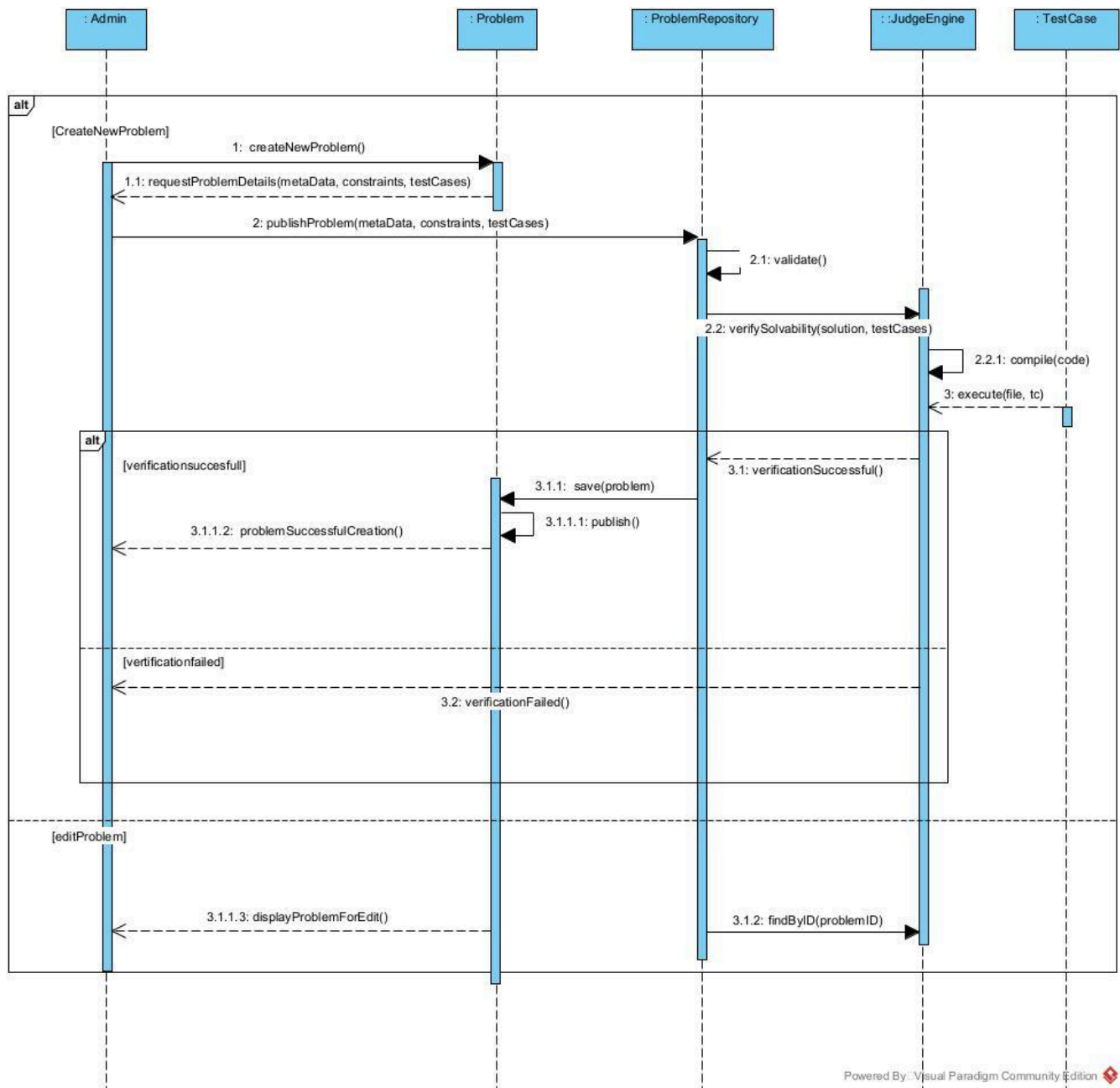


Figure 2: Domain Model

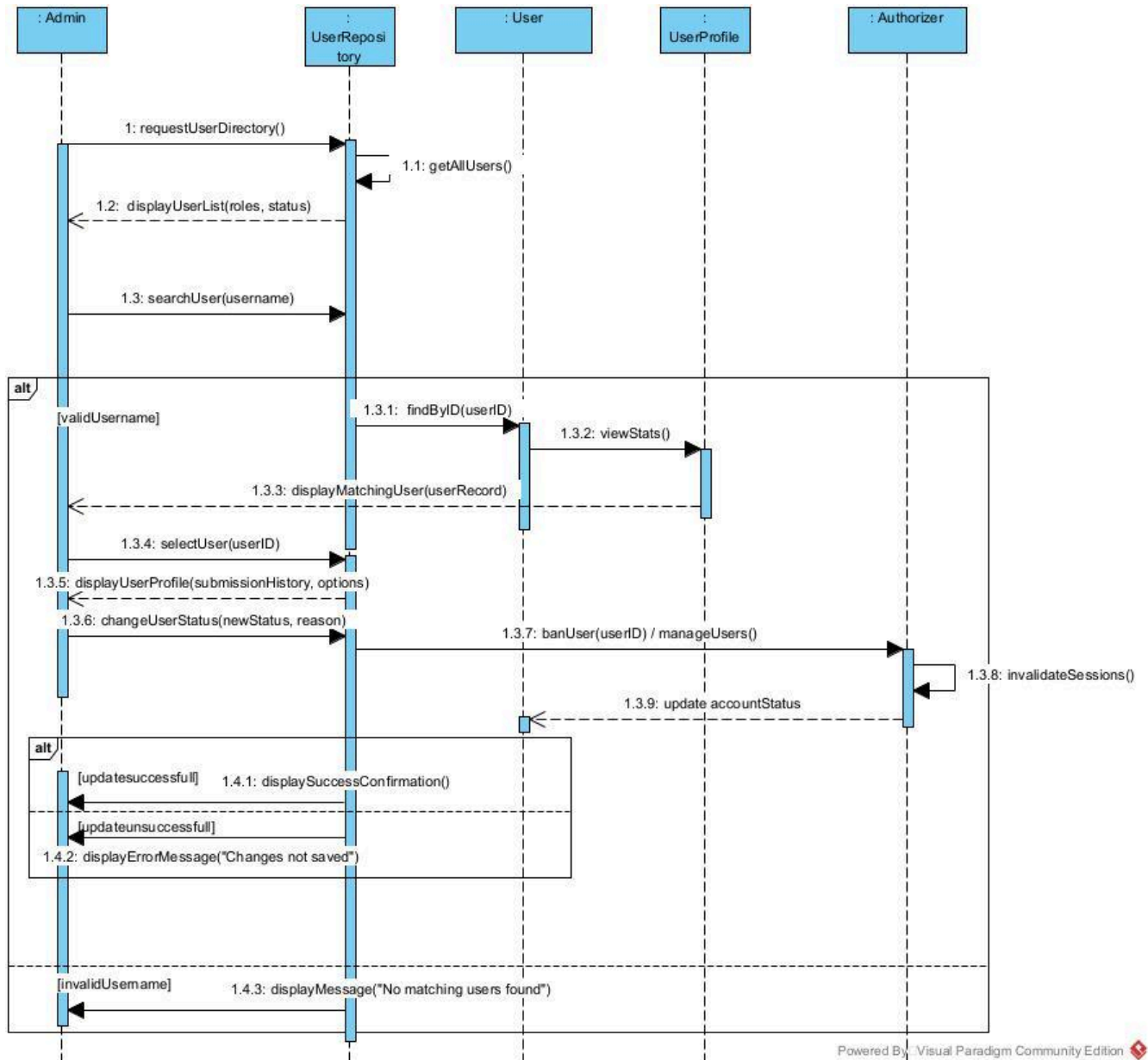
5. System Sequence Diagram

Create/Edit Problems

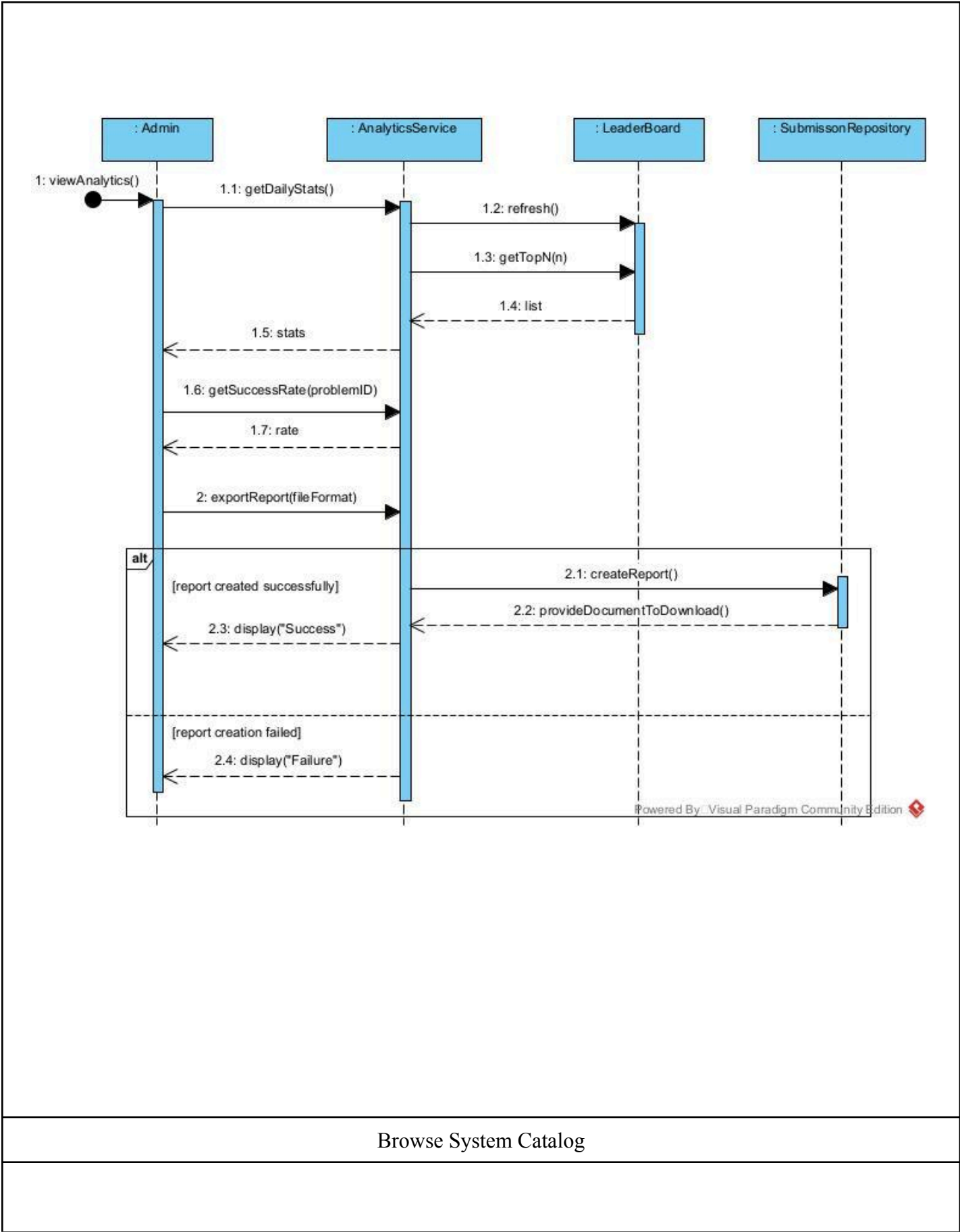


Powered By: Visual Paradigm Community Edition

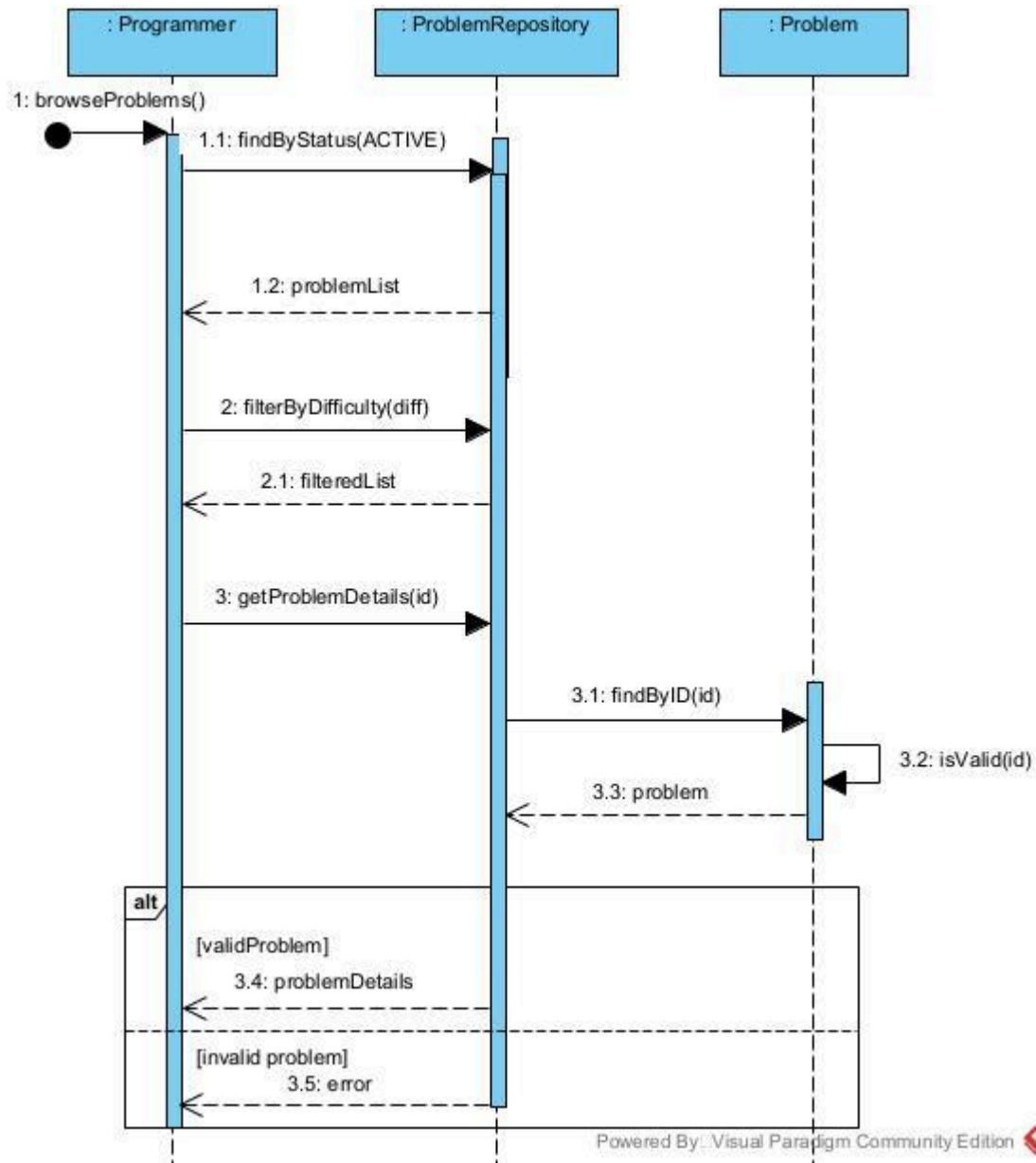
Manage Users



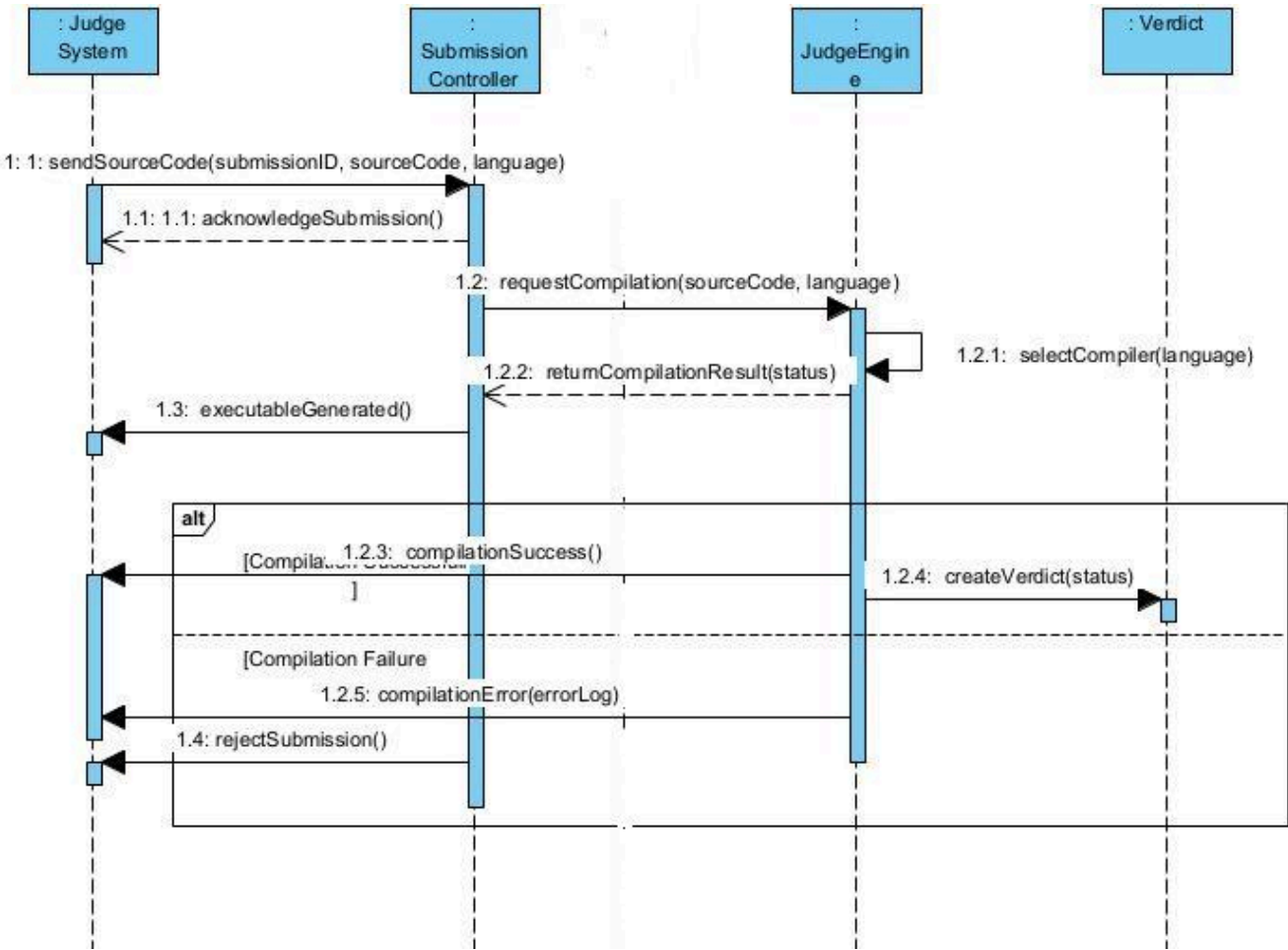
View System Analytics



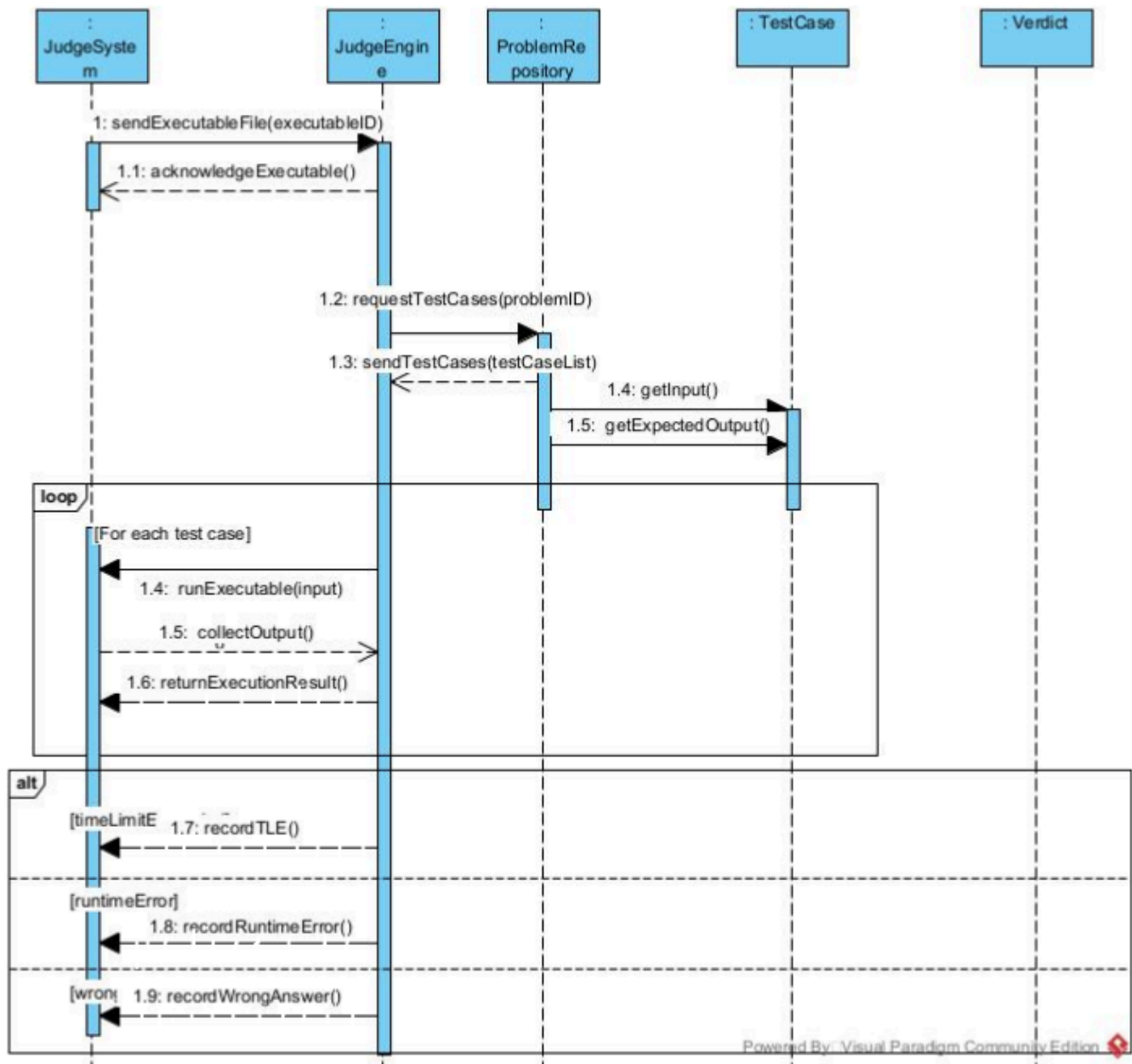
Browse System Catalog



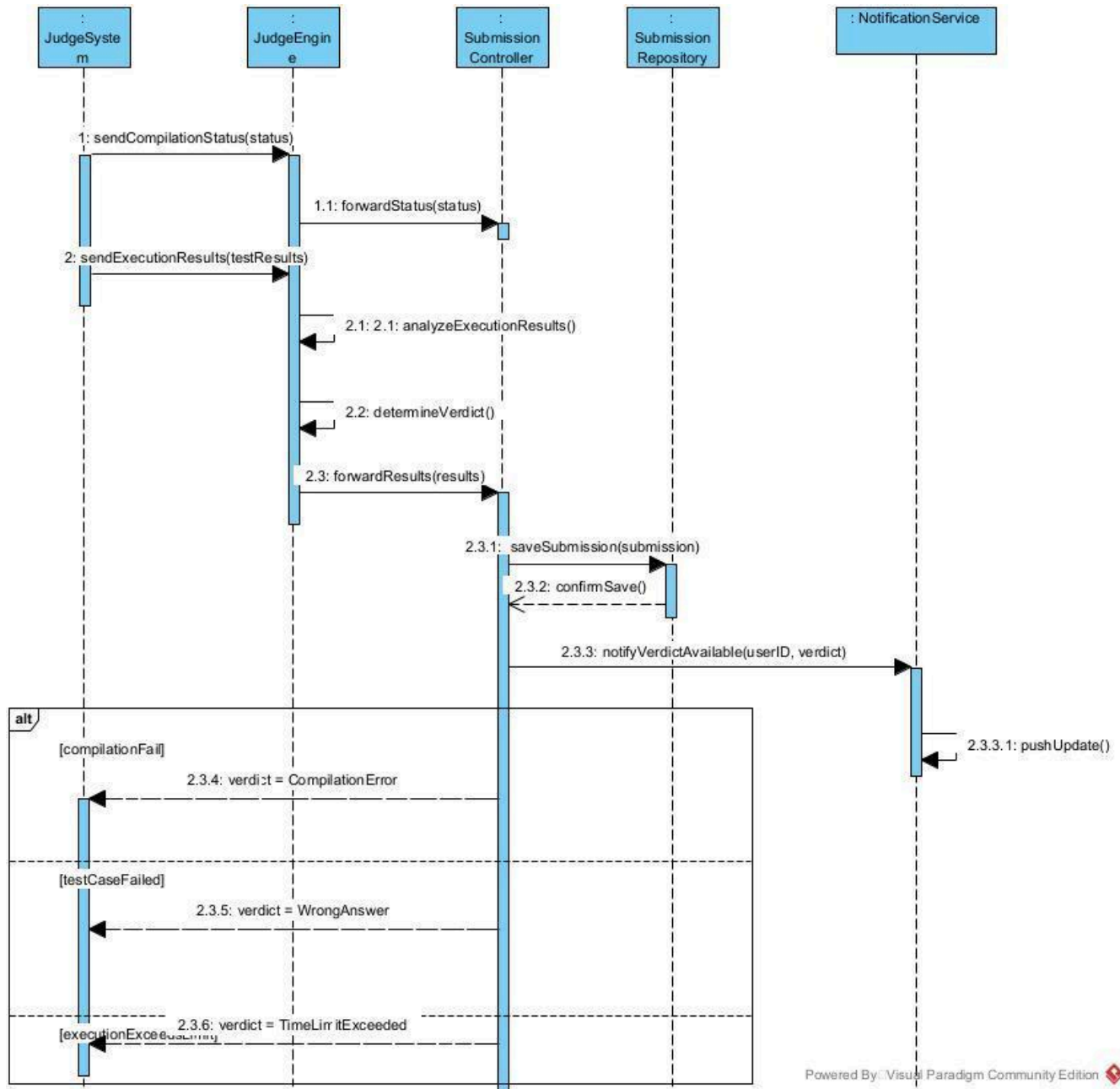
Compile Code



Execute Test Cases

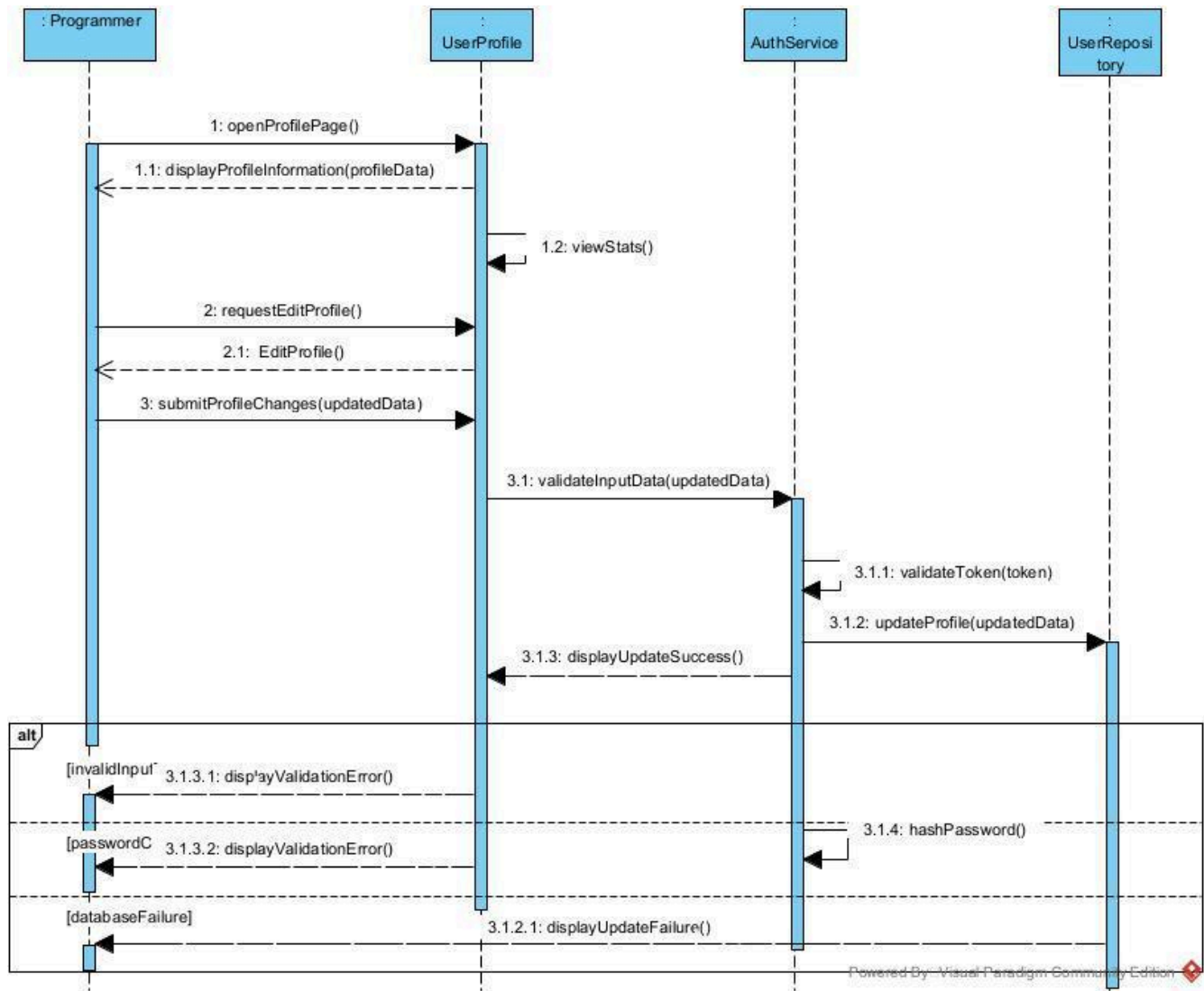


Return Verdict

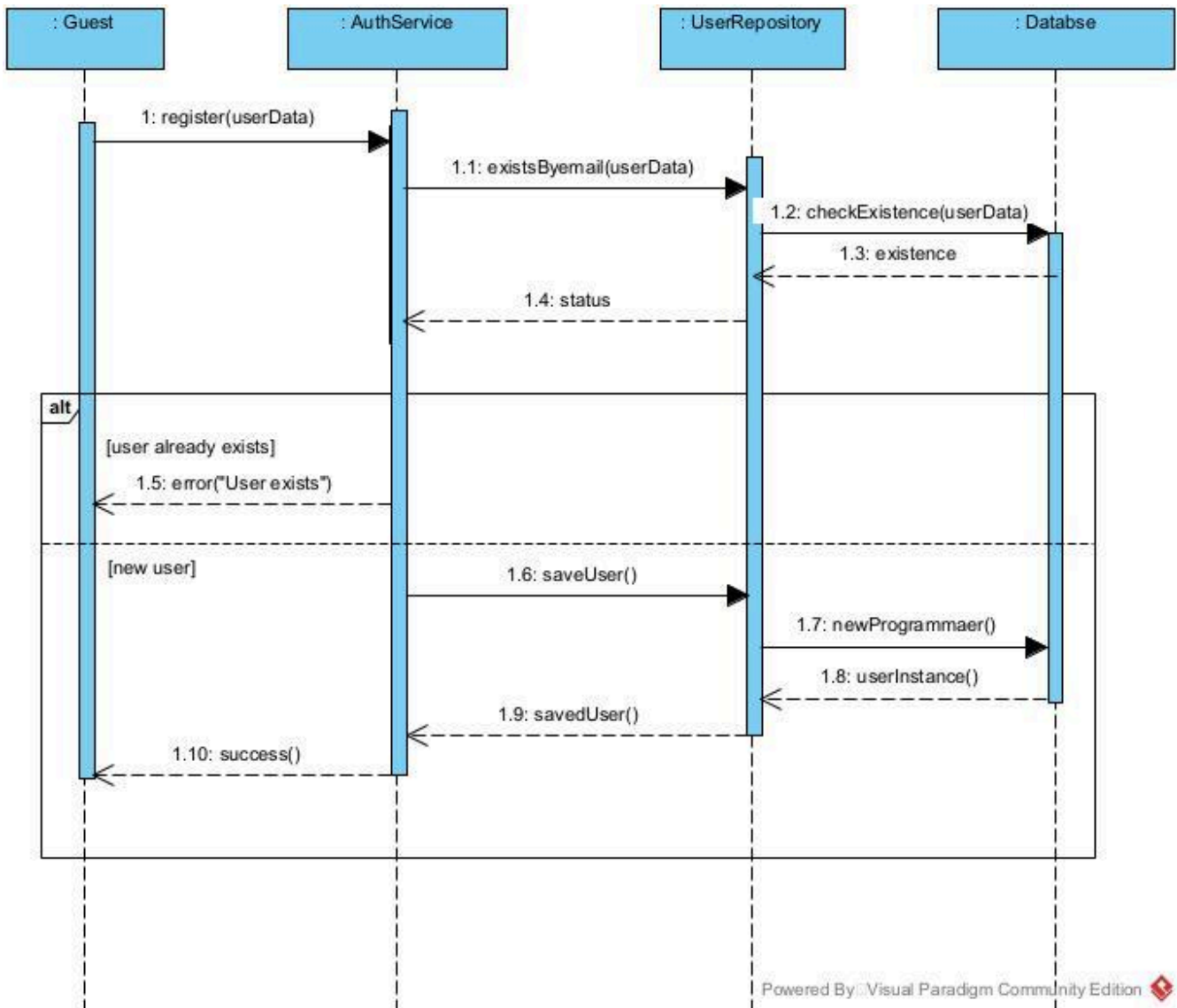


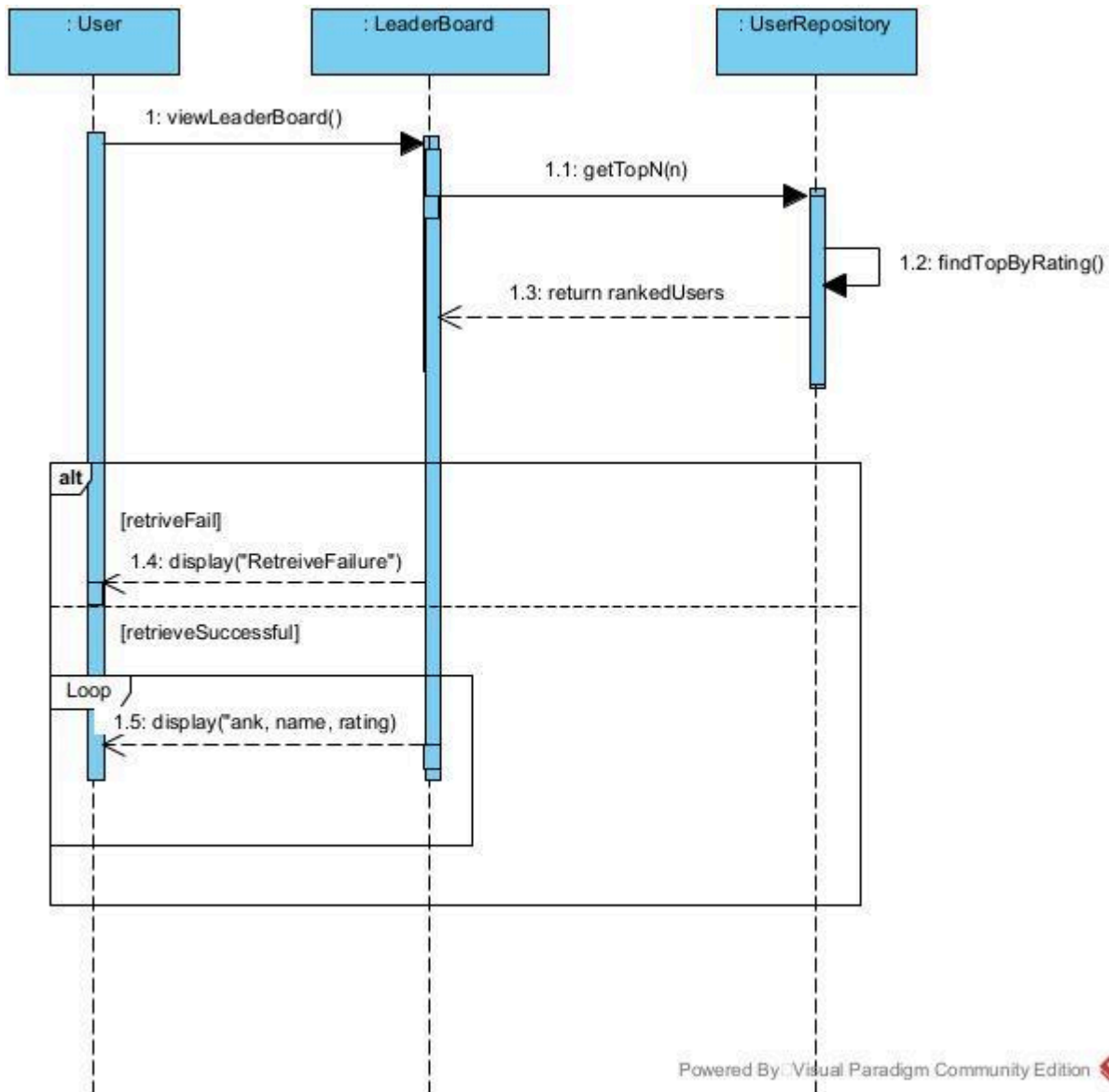
Powered By: Visual Paradigm Community Edition

Manage Profile

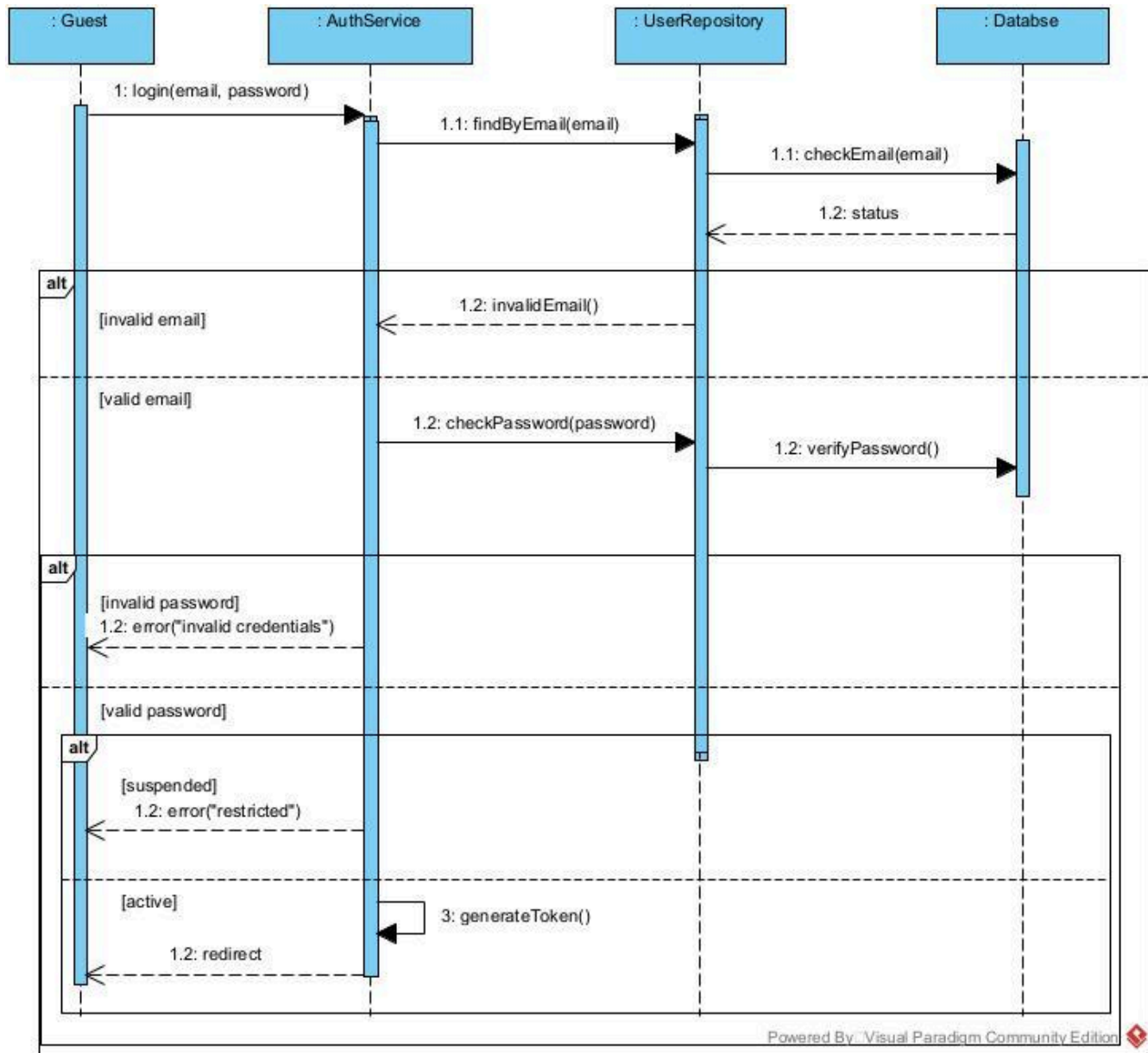


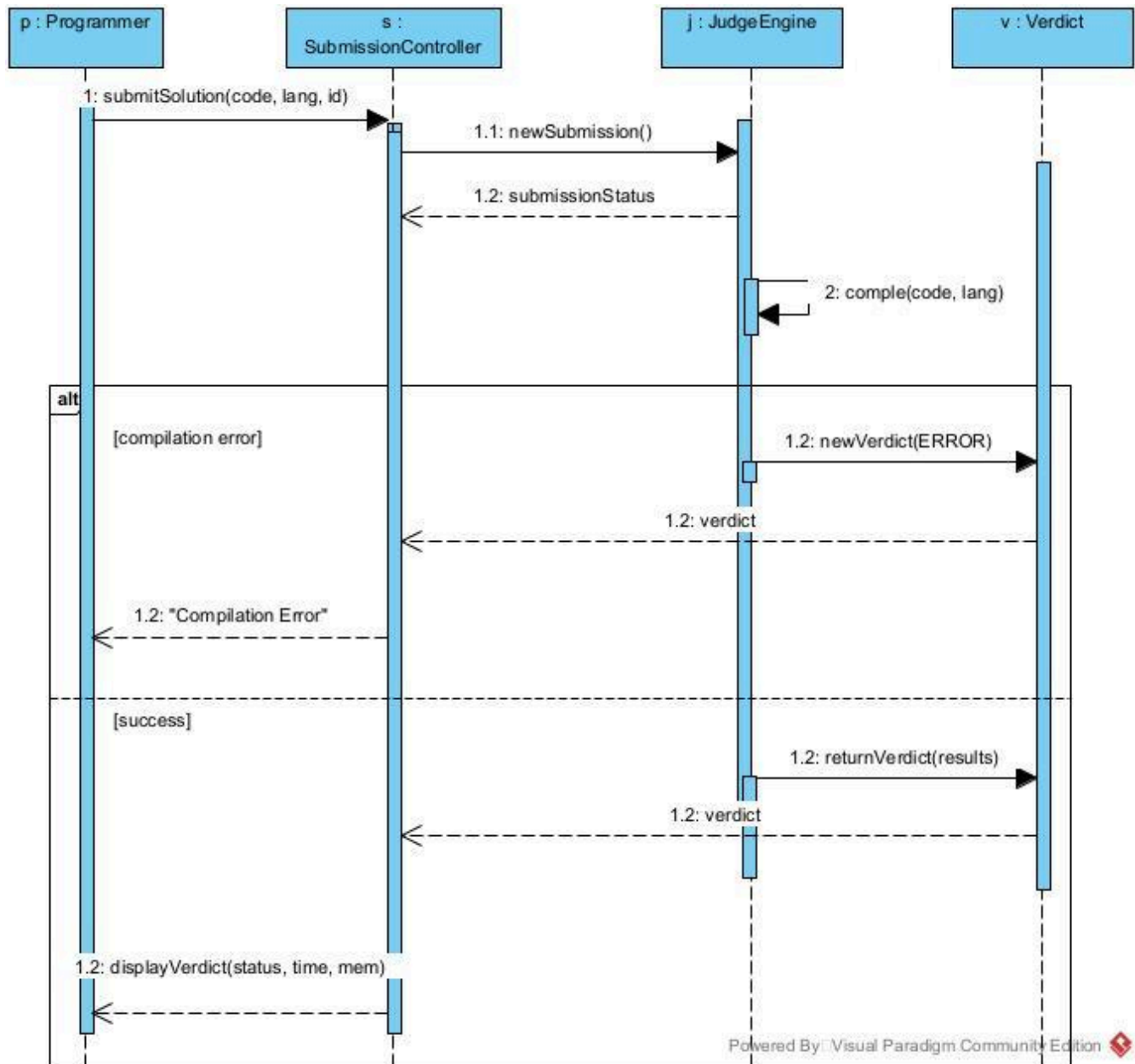
Register Account

[View Public Leaderboard](#)

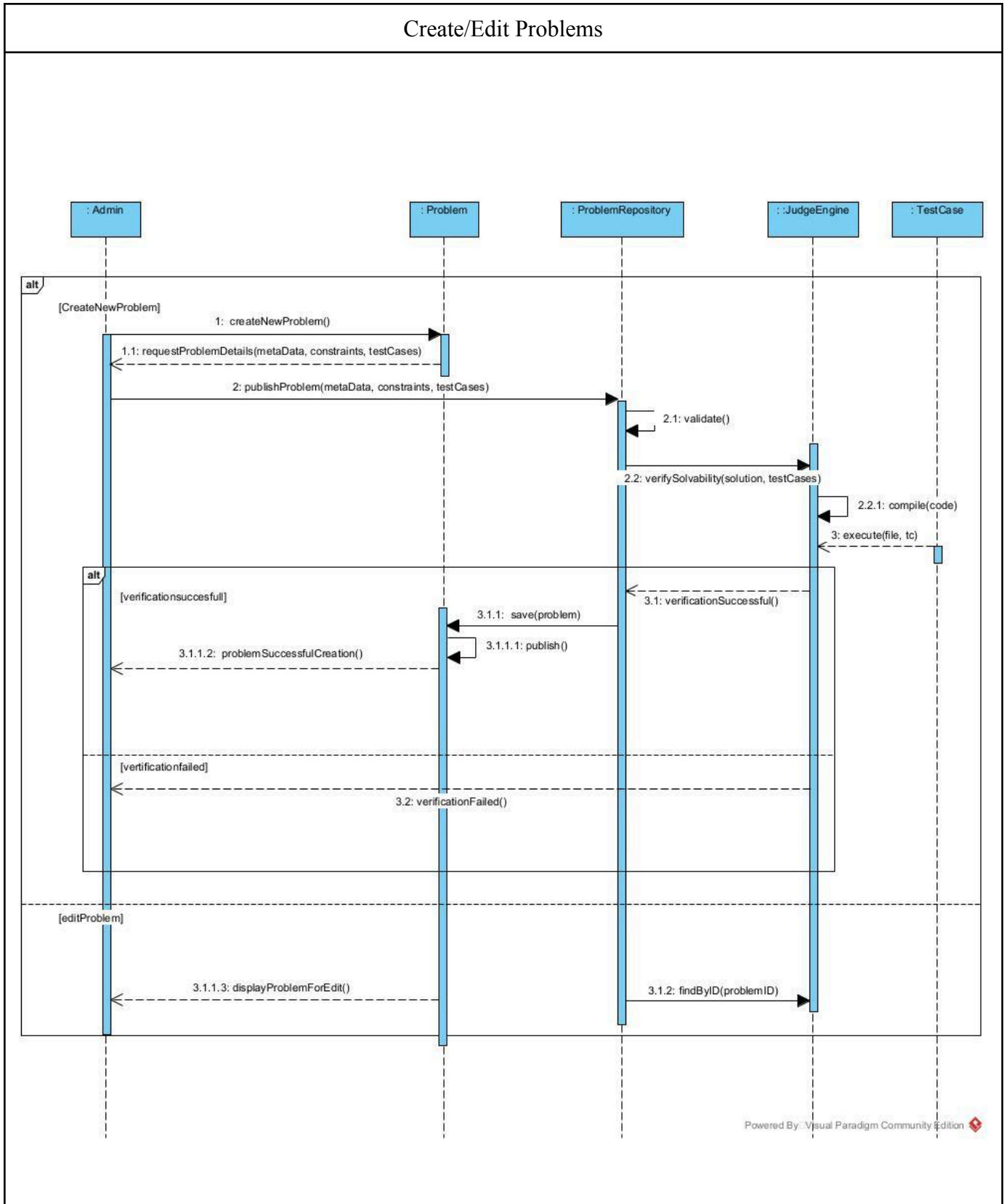


Login

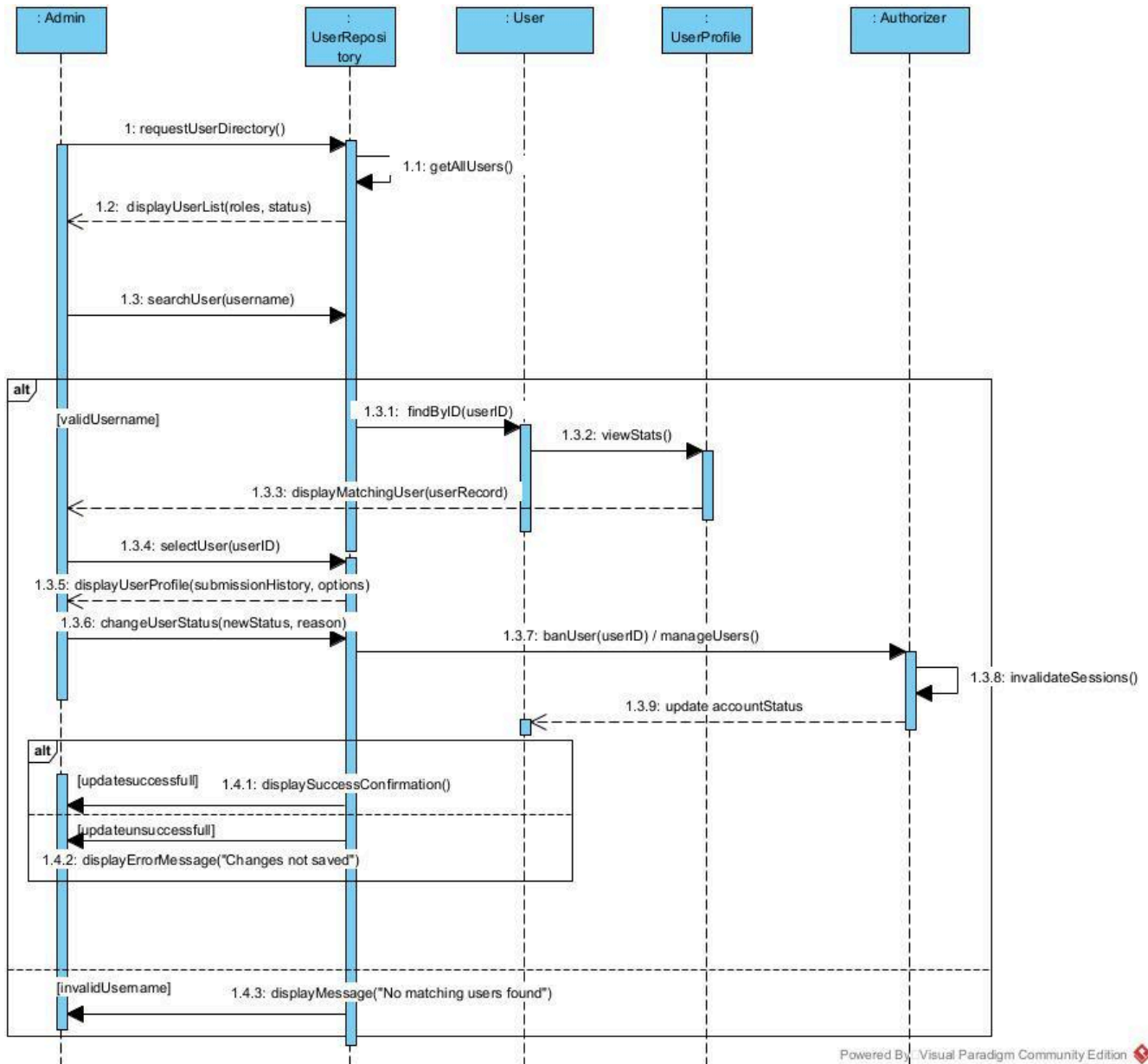
[Submit Code Solution](#)



6. Sequence Diagram

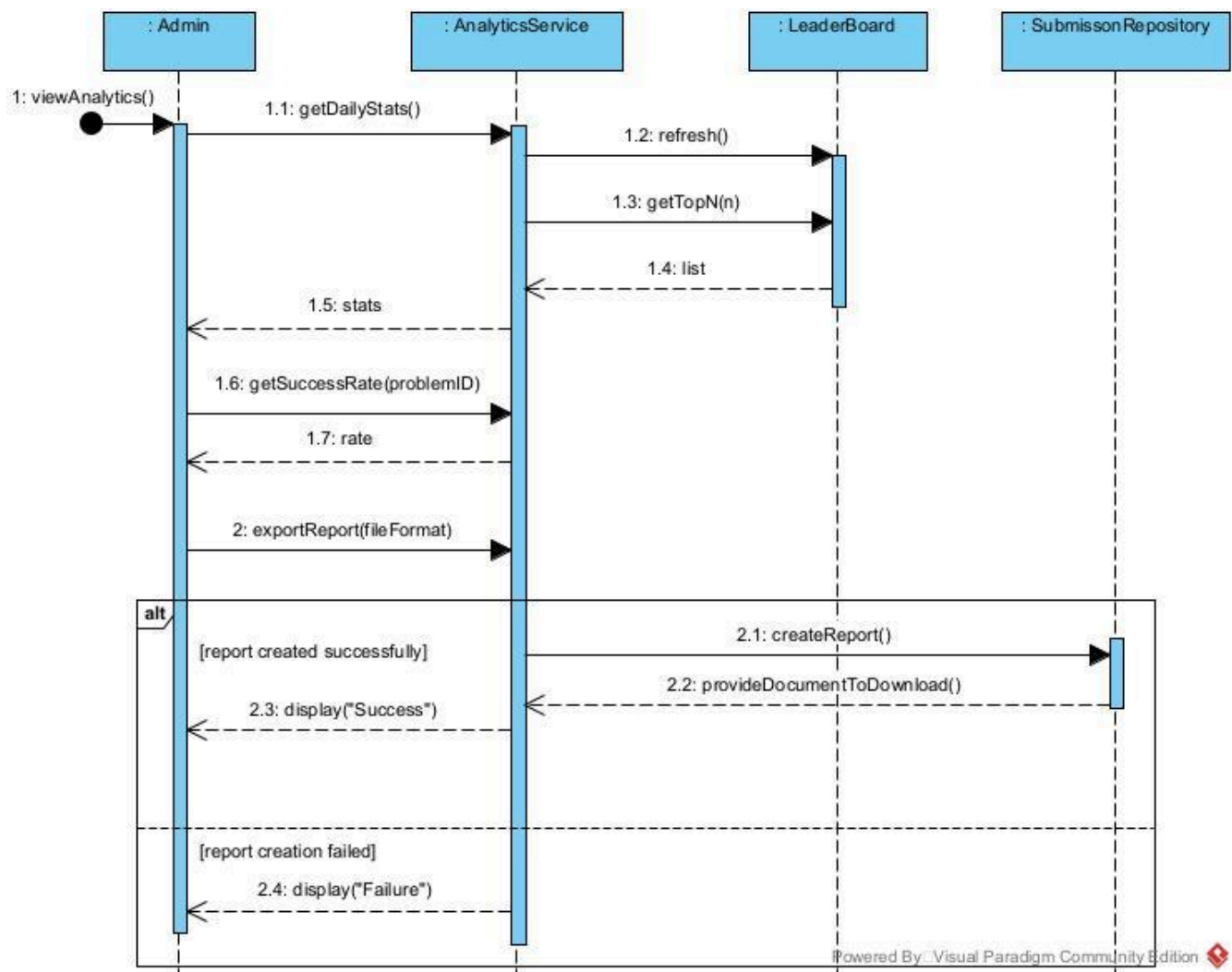


Manage Users



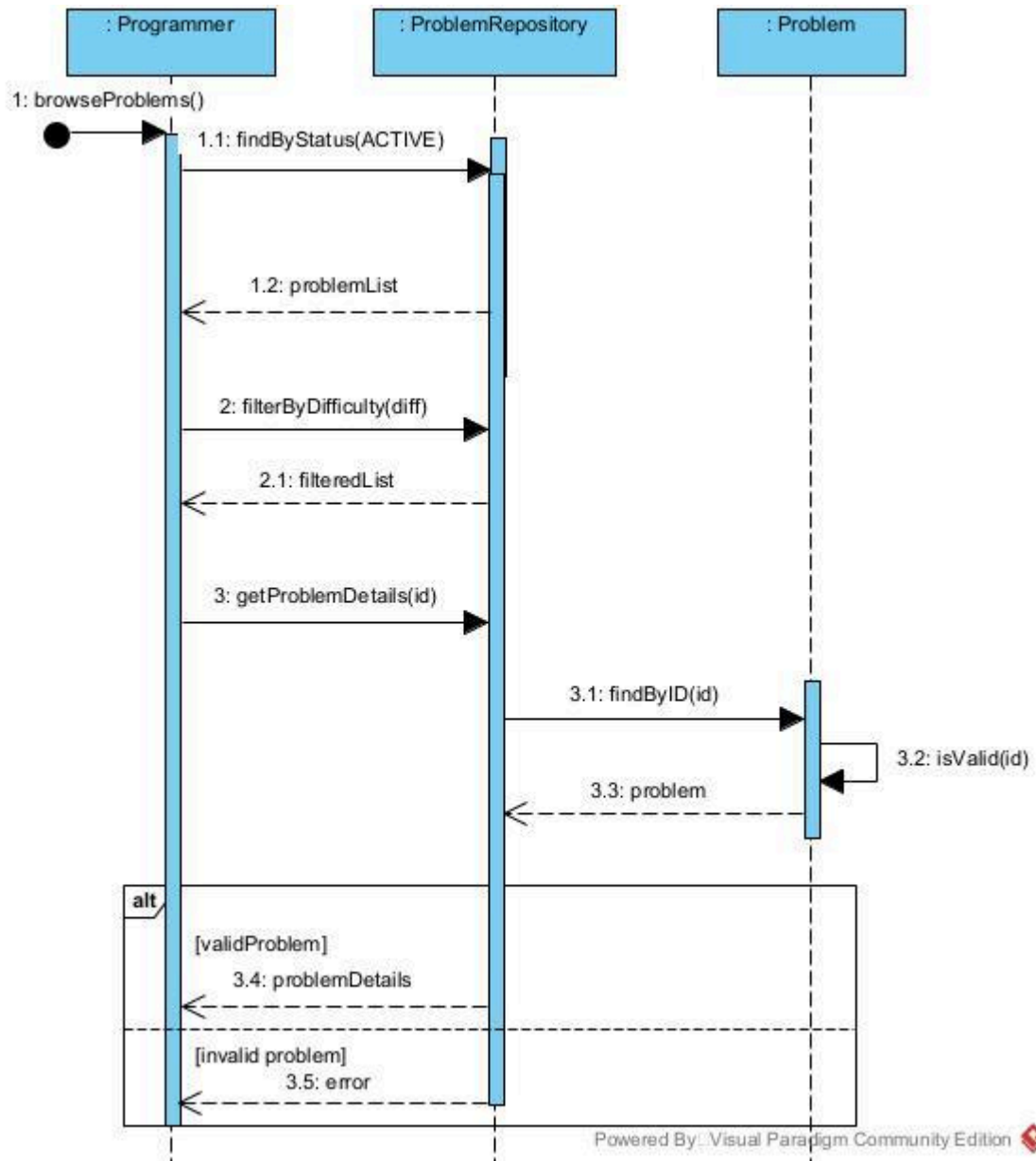
Powered By: Visual Paradigm Community Edition

View System Analytics

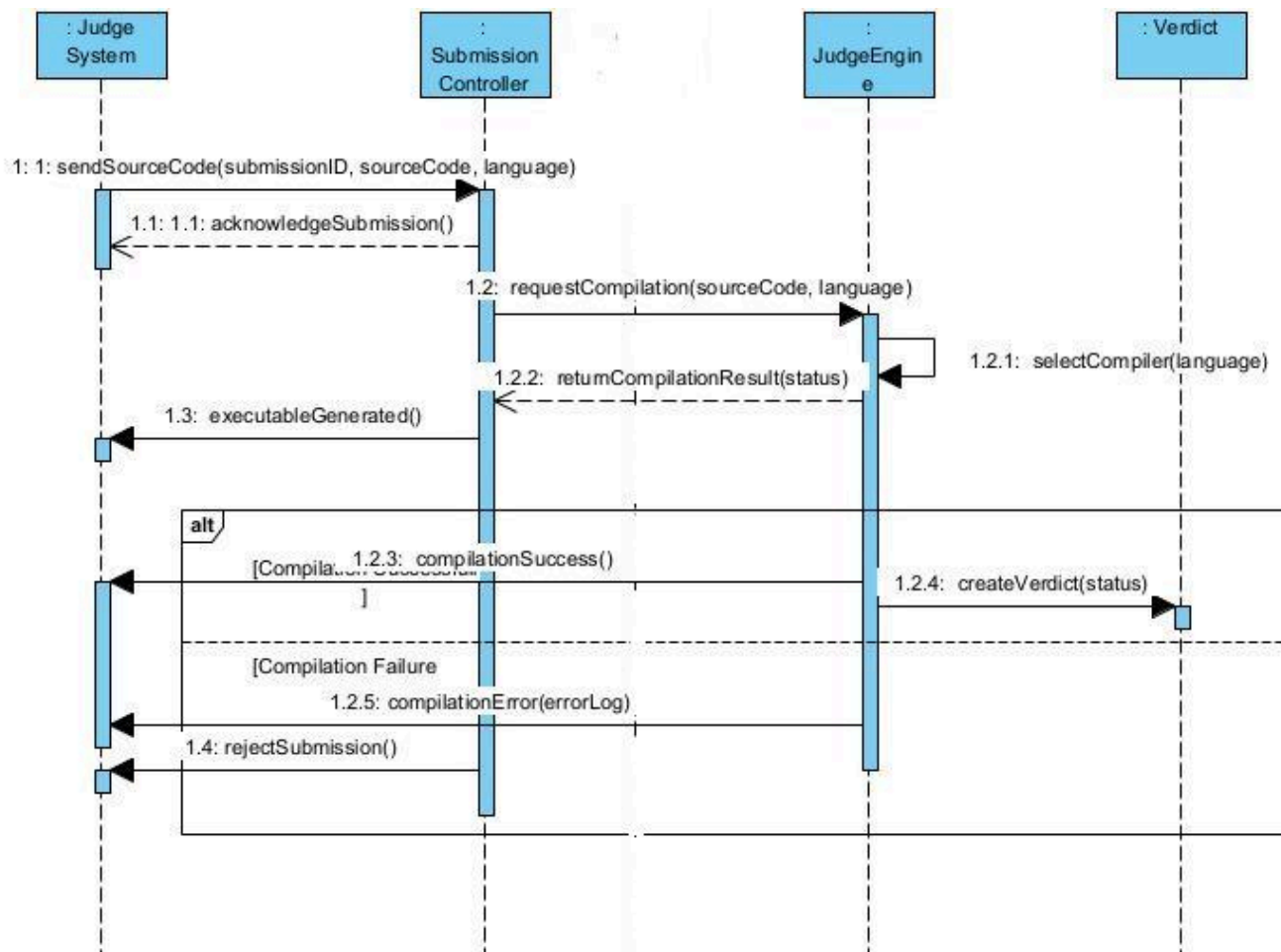


Powered By: Visual Paradigm Community Edition

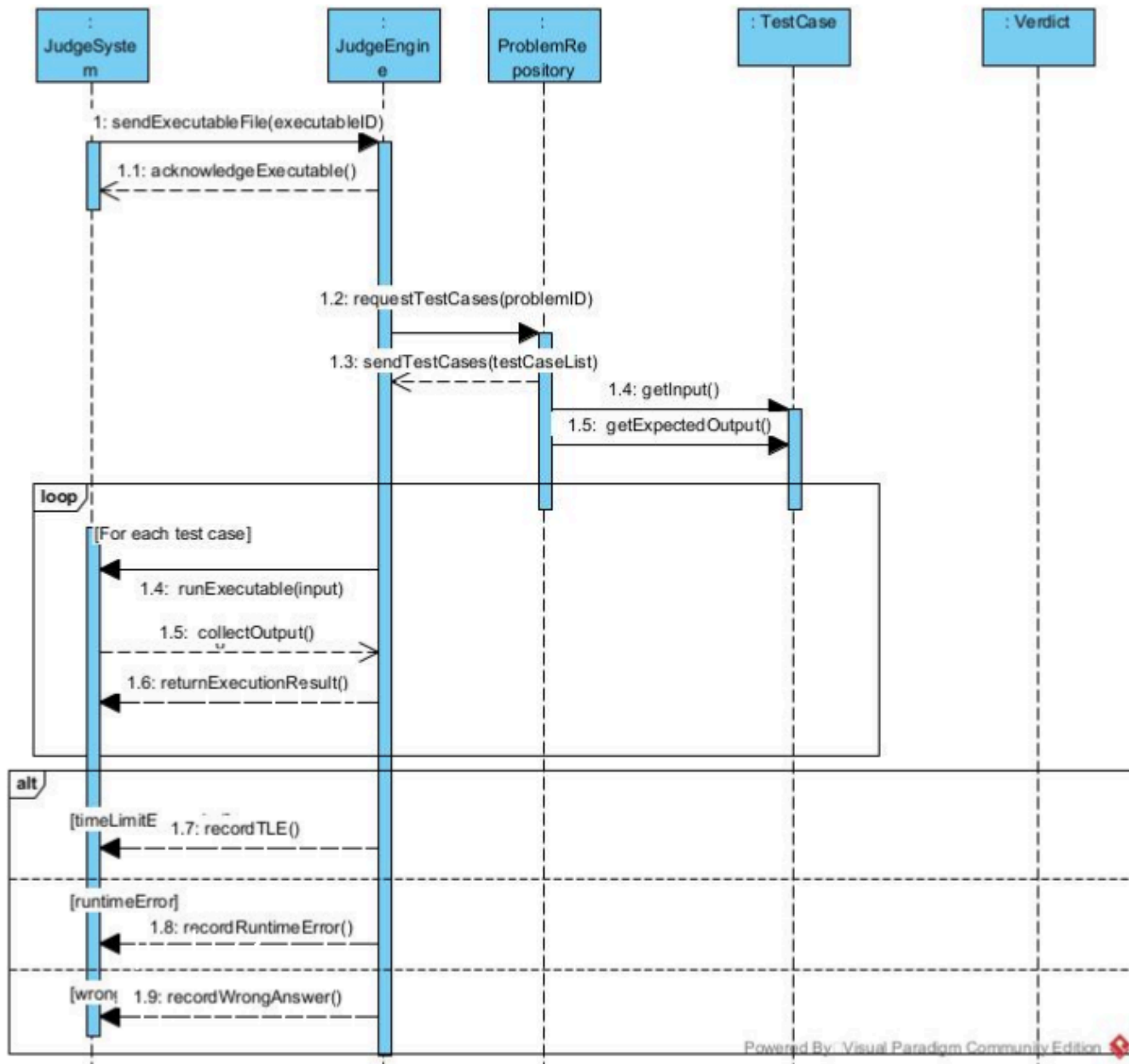
Browse System Catalog



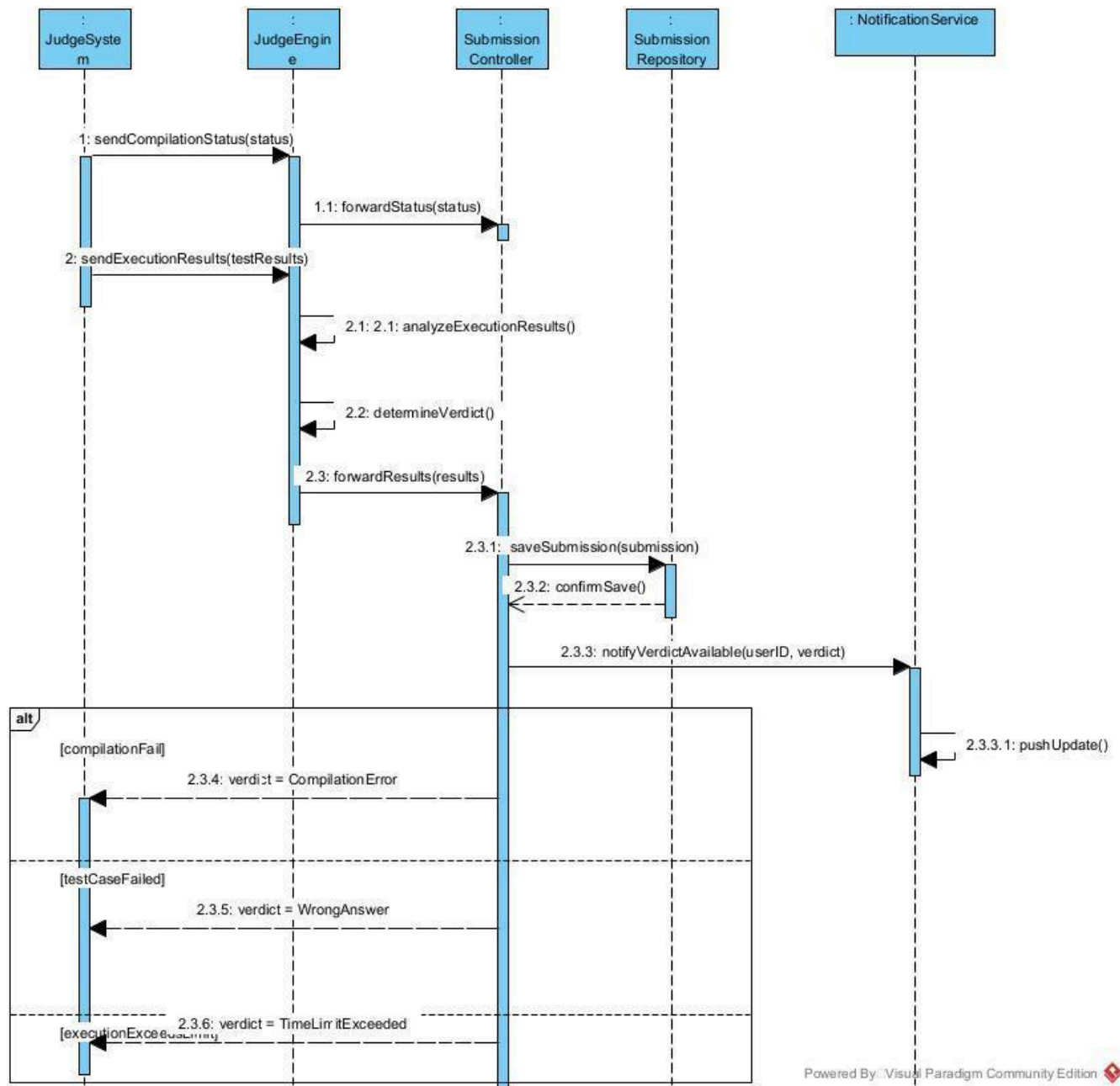
Compile Code



Execute Test Cases

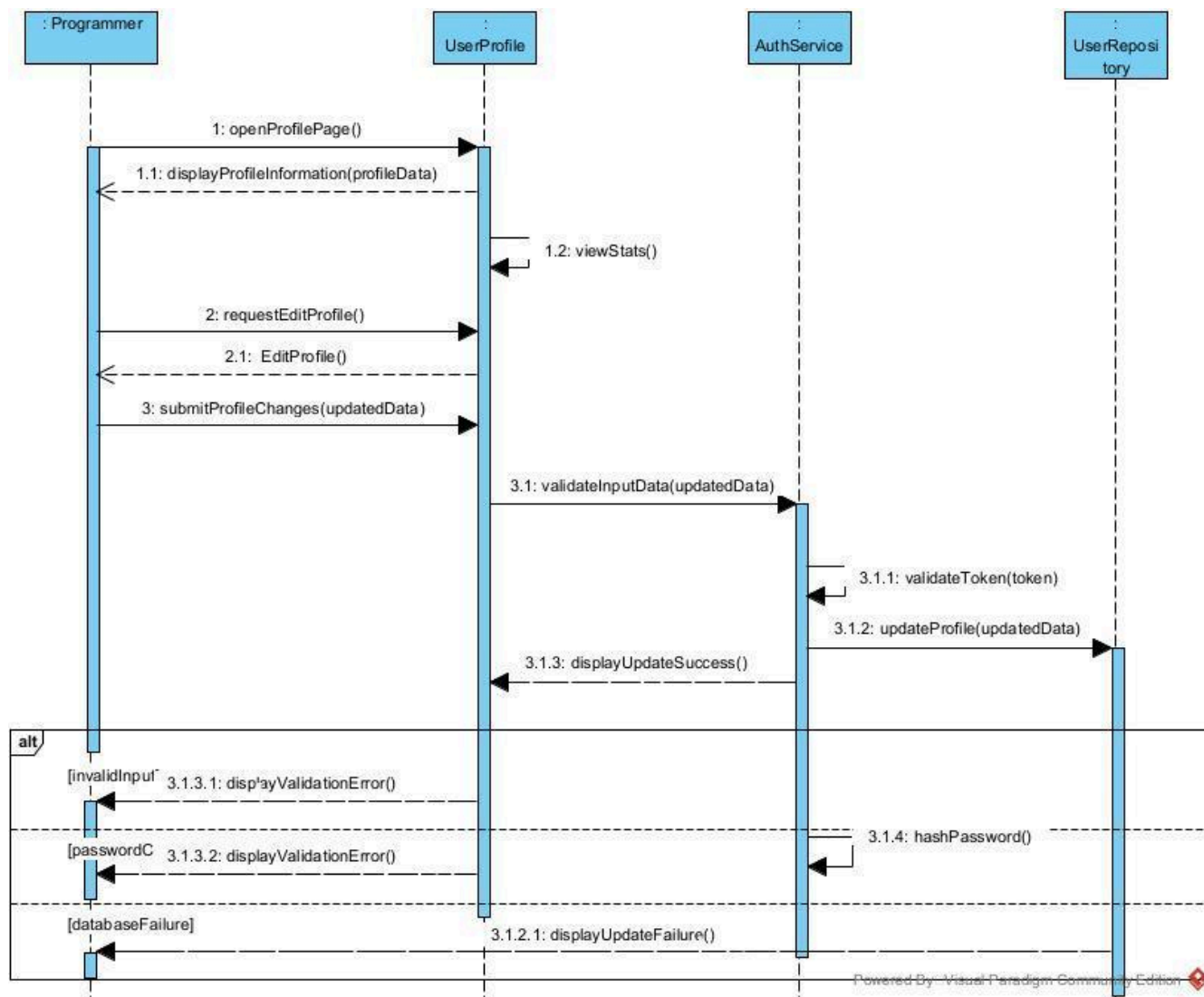


Return Verdict



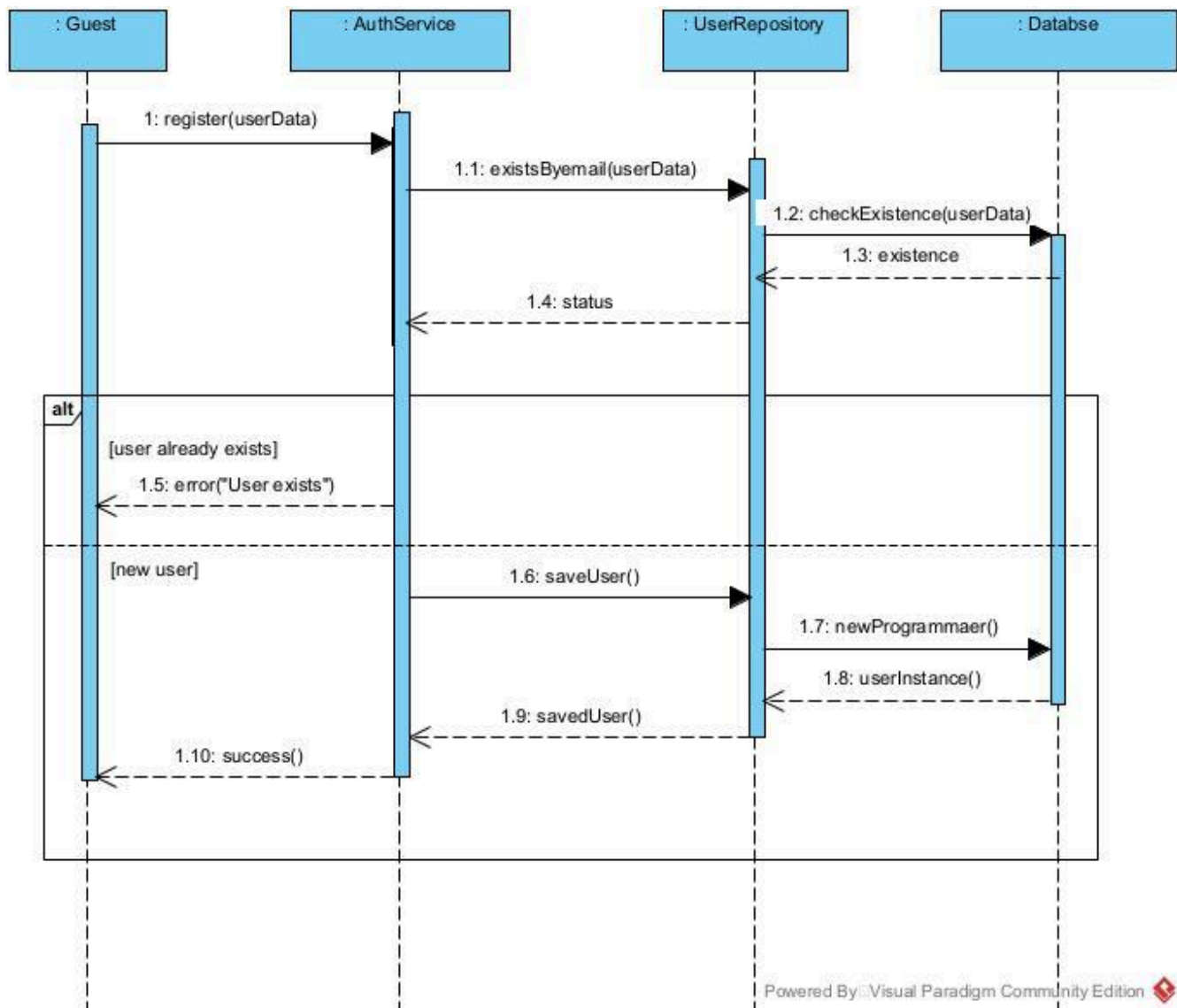
Powered By: Visual Paradigm Community Edition

Manage Profile

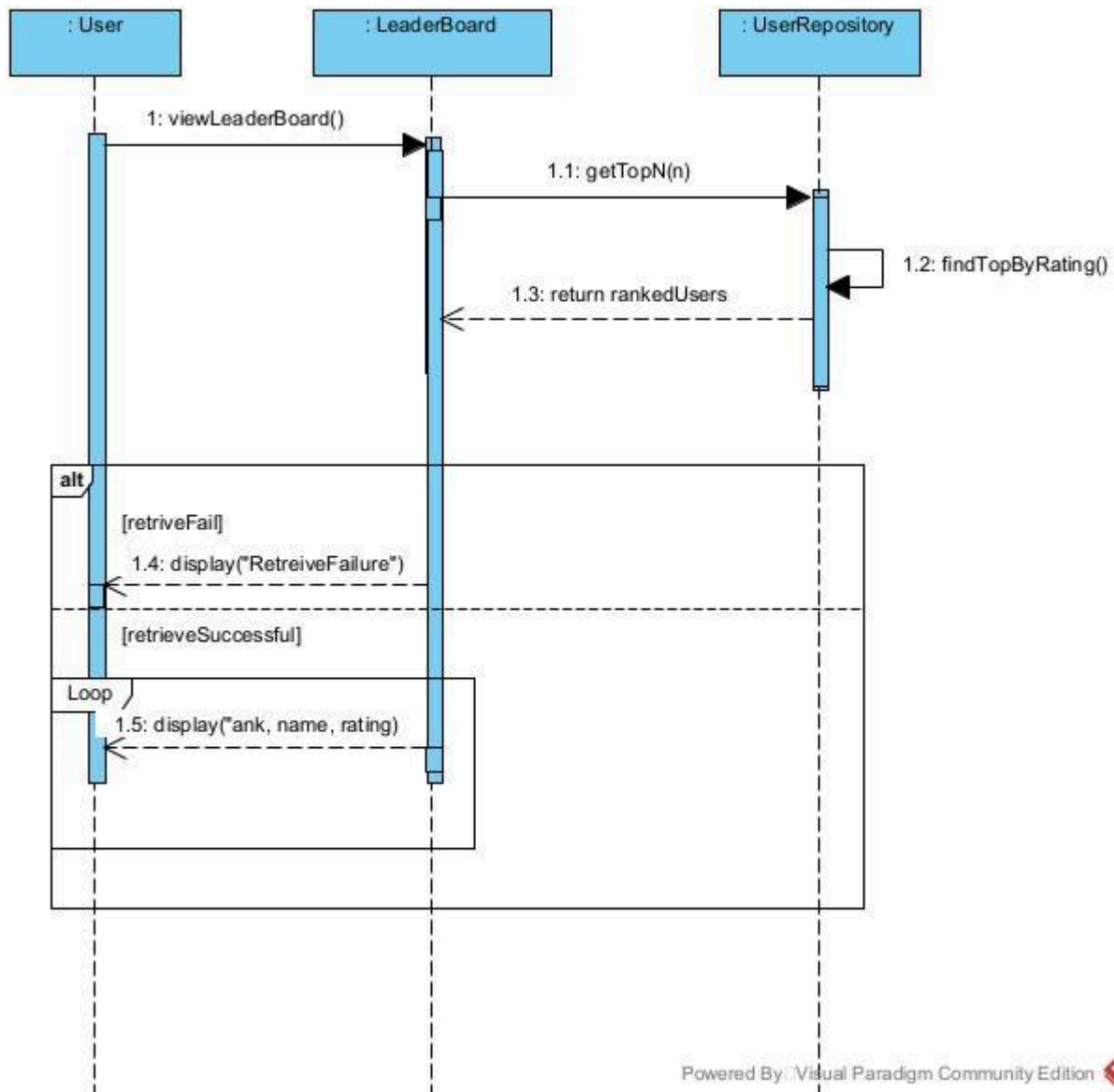


Powered By: Visual Paradigm Community Edition

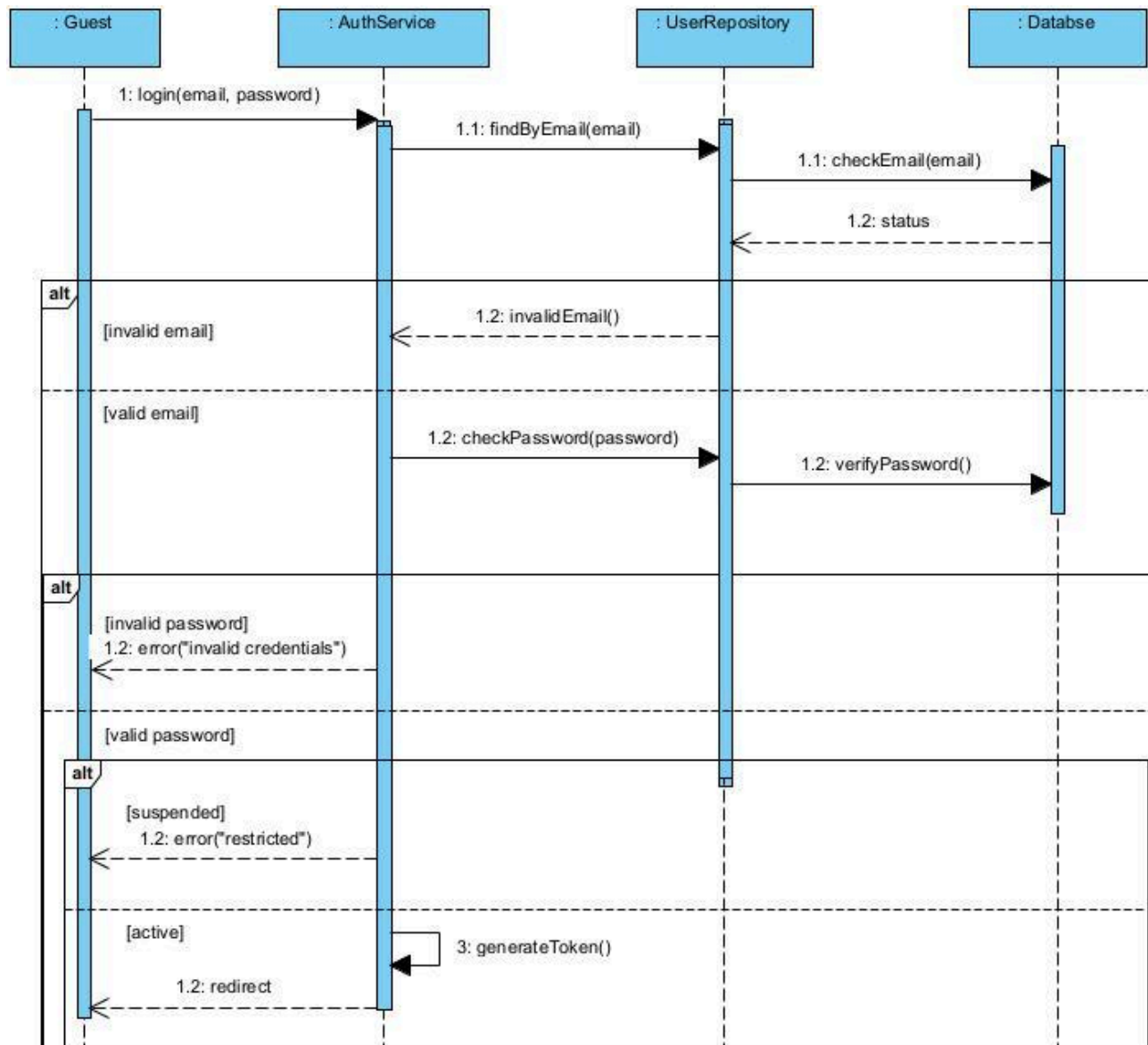
Register Account



View Public Leaderboard



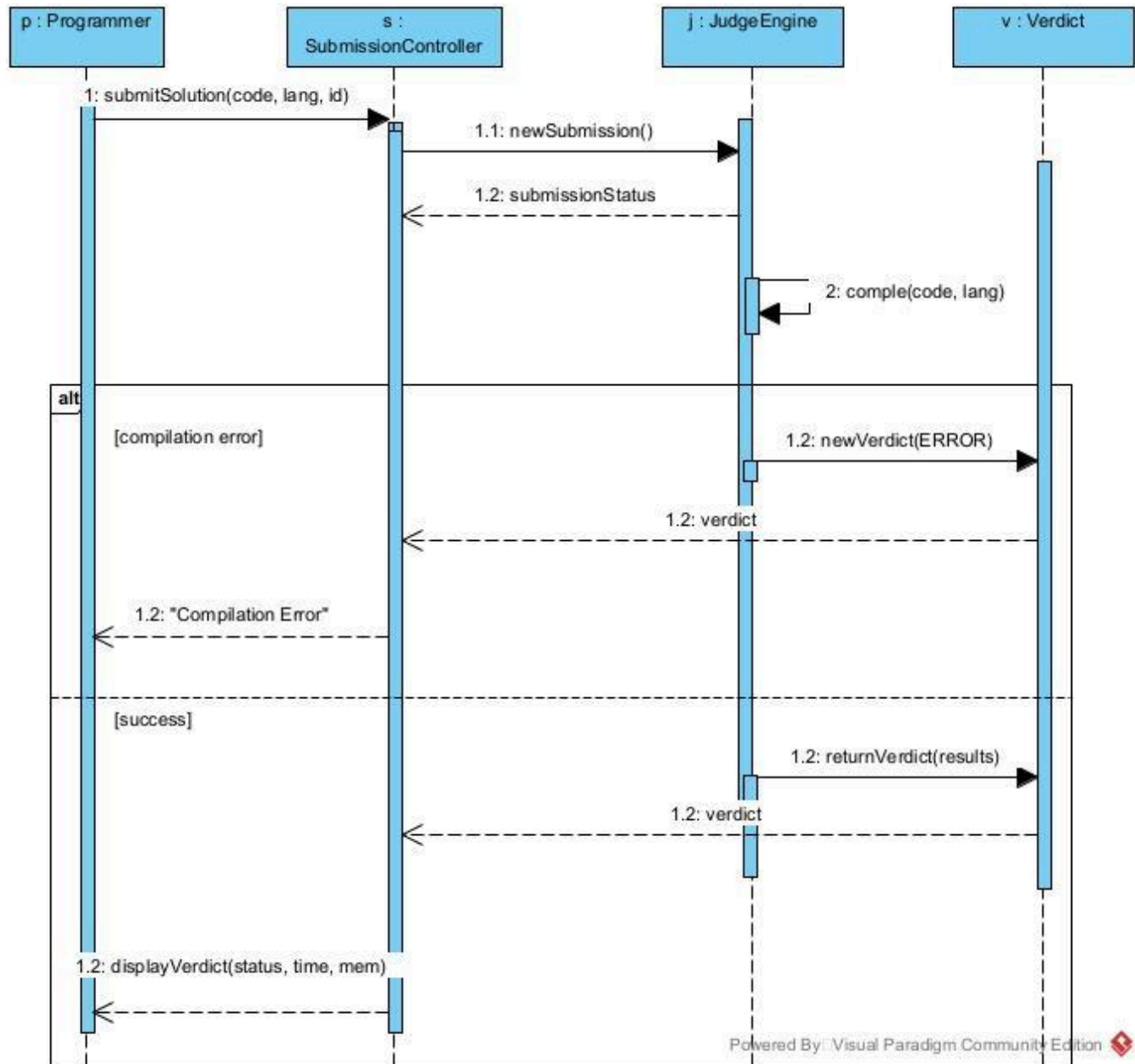
Login



Powered By Visual Paradigm Community Edition



Submit Code Solution



7. Class Diagram

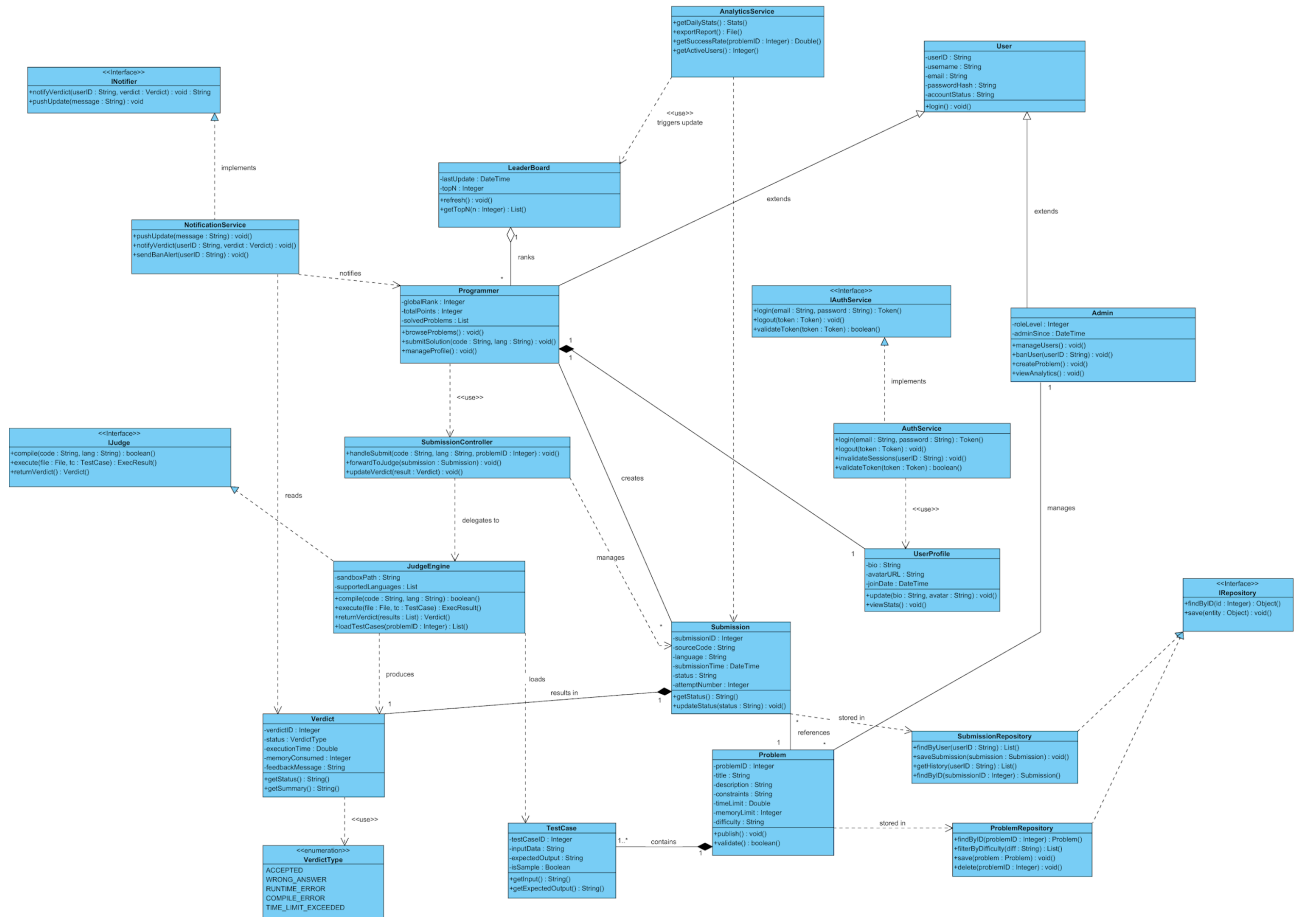


Figure 3: Class Diagram

8. Package Diagram

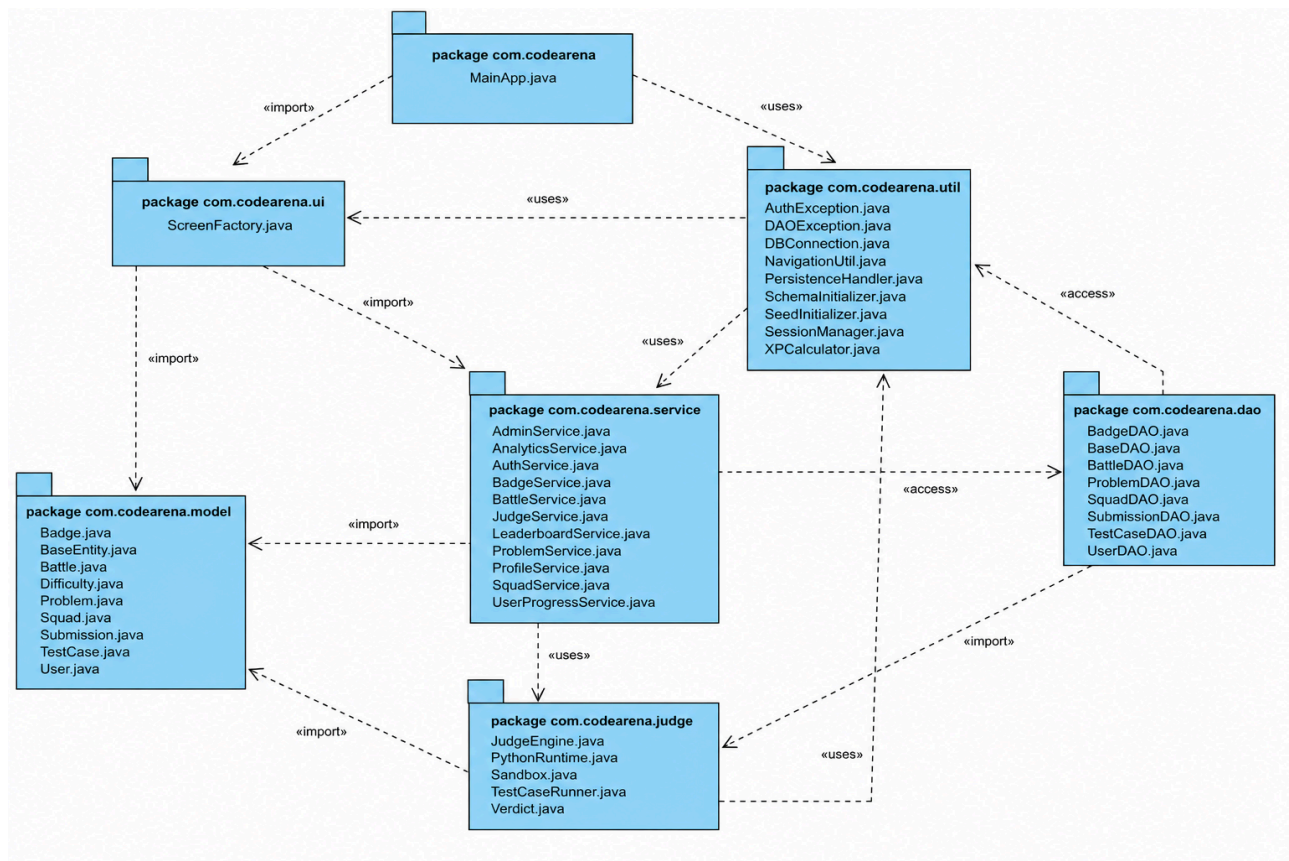
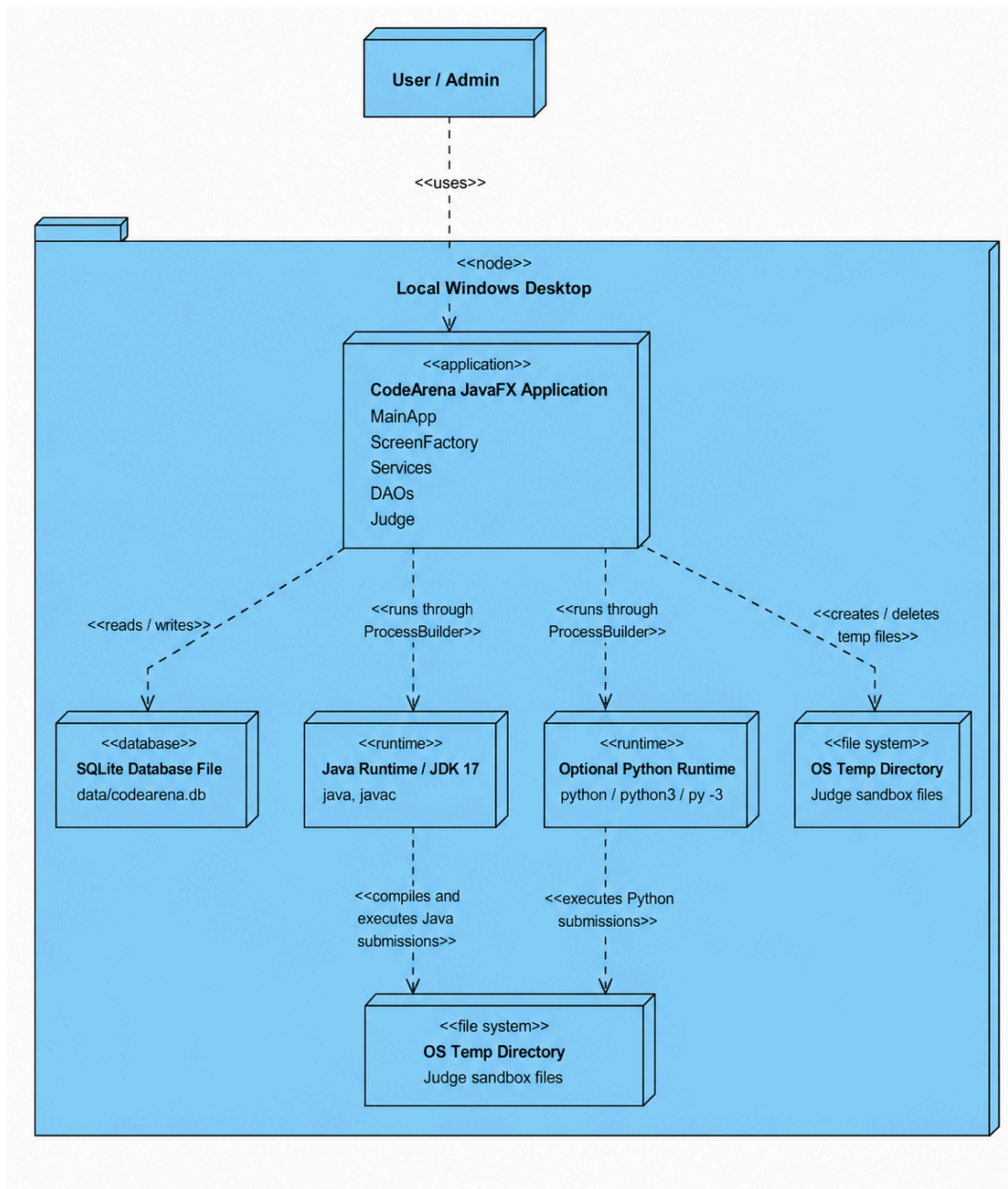


Figure 4: Package Diagram

The package diagram presents the production packages and their relationships. The main packages are com.codearena, ui, service, dao, model, judge, and util. Relationships use import, access, and uses dependencies.

9. Deployment Diagram

CodeArena is deployed as a local desktop application. The user launches the JavaFX client on a local machine. The application accesses a local SQLite database and local Java/Python runtimes for judging submissions.



10. Database Design and Persistence

CodeArena uses SQLite as its local database. By default, the database is stored at `data/codearena.db`. When the database is missing, startup creates schema objects from `src/main/resources/data/schema.sql` and inserts seed/demo data from `src/main/resources/data/seed.sql` and `SeedInitializer`.

- DAO classes isolate SQL operations from UI and services.
- `PersistenceHandler` coordinates schema initialization and seeding.
- Core data includes users, problems, test cases, submissions, badges, battles, battle participants, squads, and squad members.

11. Testing and Validation

Automated JUnit tests are stored under `src/test/java` and use a separate test database at `target/test-data/codearena-test.db`. The tests verify service behavior, persistence initialization, and local judge behavior without modifying the real application database.

- `ServiceIntegrationTest` covers authentication, problem filtering, admin validation, badge awarding, user progress, and squads.
- `JudgeEngineIntegrationTest` covers AC, WA, CE, RE, TLE, Python behavior when available, and sandbox cleanup.
- `PersistenceInitializationTest` verifies fresh schema and seed initialization.
- The latest verification command used during development was `mvn -q test`, and the suite passed.

12. Conclusion

CodeArena demonstrates a complete Software Design and Analysis project with a layered Java architecture, local persistence, a JavaFX user interface, local judging, gamified progress, battle workflows, admin tools, and automated validation. The application is suitable for final demonstration and can be extended further with packaged installers, more problem content, and richer analytics.